A MAJOR PROJECT

On

MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION

Submitted

In partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (Data Science)

By

Name of team members	Rollno
Nagarjunapu Sai Vishwa Teja	21641A67D4
Urukonda Shruti	21641A67G6
Vuppula Charan Tej	21641A67J5

Under the Guidance of

Mrs. S. LakshmiLalitha

Associate Professor, Department of CSE (Data Science).

Department of Computer Science & Engineering (Data science)

Vaagdevi College of Engineering

(UGC Autonomous, Accredited by NAAC with "A")
Bollikunta, Khila Warangal (Mandal), Warangal Urban – 506005(T.S)

(2021-2025)

VAAGDEVI COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

(UGC Autonomous, Accredited by NBA, Accredited by NAAC with "A")

Bollikunta, Khila Warangal (Mandal), Warangal Urban –506005(T.S)

CERTIFICATE

This is to certify that the major project entitled "***MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION"*** is submitted by **N. Sai Vishwa Teja 21641A67D4 , U. Shruti 21641A67G6, V. Charan Tej 21641A67J5** in partial fulfillment of the requirements for the award of the Degree in Bachelor of Technology in Computer Science and Engineering(Data Science) during the academic year 2024-2025.

Project Guide:
Mrs.S.LakshmiLalitha

Head of the Department:
Dr. Ayesha Banu

External Examiner

DECLARATION

We declare that the work reported in the project entitled "MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION" is a record of work done by us in the partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering(DataScience), VAAGDEVI COLLEGE OF ENGINEERING (Autonomous), Affiliated to JNTUH, Accredited By NBA, under the guidance of **Mrs.S.LakshmiLalitha**, Associate Professor of CSE (DS). We hereby declare that this project work bears no resemblance to any other project submitted at Vaagdevi College of Engineering or any other university/college for the award of the degree.

Nagarjunapu Sai Vishwa Teja (21641A67D4)

Urukonda Shruti (21641A67G6)

Vuppula Charan Tej (21641A67J5)

ACKNOWLEDGEMENT

The development of the project though it was an arduous task, it has been made by the help of many people. We are pleased to express our thanks to the people whose suggestions, comments, criticisms greatly encouraged us in betterment of the project.

We would like to express our sincere gratitude and indebtedness to my project Guide **Mrs. S. LakshmiLalitha** Assistant Professor, CSE (DS) for her valuable suggestions and interest throughout the completion of this project.

We are also thankful to Head of the Department **Dr. Ayesha Banu**, Associate Professor, CSE (DS) for providing excellent support in completing the project successfully.

We would like to express our sincere thanks and profound gratitude to **Dr. K. Prakash kumar**, Principal of Vaagdevi College of Engineering, for his support, guidance and encouragement in the course of our project.

We are also thankful to Project Coordinators, for their valuable suggestions, encouragement and motivations for completing this project successfully.

We are thankful to all other faculty members for their encouragement.

Finally, we would like to take this opportunity to thank our family for their support through the work. We sincerely acknowledge and thank all those who gave directly or indirectly their support in completion of this work.

Nagarjunapu Sai Vishwa Teja (21641A67D4)

Urukonda Shruti (21641A67G6)

Vuppula Charan Tej (21641A67J5)

ABSTRACT

With cyber threats increasing at an alarming rate, security breaches have surged by 67% over the past five years, and cybercrime is expected to cost the world $10.5 trillion annually by 2025. Traditional manual threat detection methods rely on rule-based systems and human expertise, which are prone to errors, slow in response, and inefficient in handling large-scale data. To address these challenges, we propose a machine learning-based behavioral analysis approach for security threat detection, leveraging deep learning for improved accuracy. The proposed method begins with data preprocessing to clean and normalize the Security Threat Dataset, which contains four attack labels: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). To handle class imbalance, we employ the ADASYN (Adaptive Synthetic Sampling) technique, which generates synthetic minority class samples, ensuring a balanced dataset. The dataset is then split into training and testing sets to evaluate model performance. We compare traditional classifiers such as Naïve Bayes and Support Vector Machine (SVM) with our proposed Convolutional Neural Network (CNN)-based model, which captures complex patterns and hierarchical representations in network traffic data. Experimental results demonstrate that CNN outperforms existing methods in terms of detection accuracy, recall, and precision, making it a robust solution for real-time security threat analysis.

TABLE OF CONTENTS

CHAPTER 1

1. INTRODUCTION

Overview

The rapid development of the IoT has brought significant growth to the global economy. However, it has also brought security risks such as leakage of core industrial data and unauthorized manipulation of interconnected terminals. Industrial firewalls are a common means of protecting the Industrial Internet of Things. However, industrial firewall technology only provides passive defense mechanisms and may not effectively prevent files and programs containing threat codes.

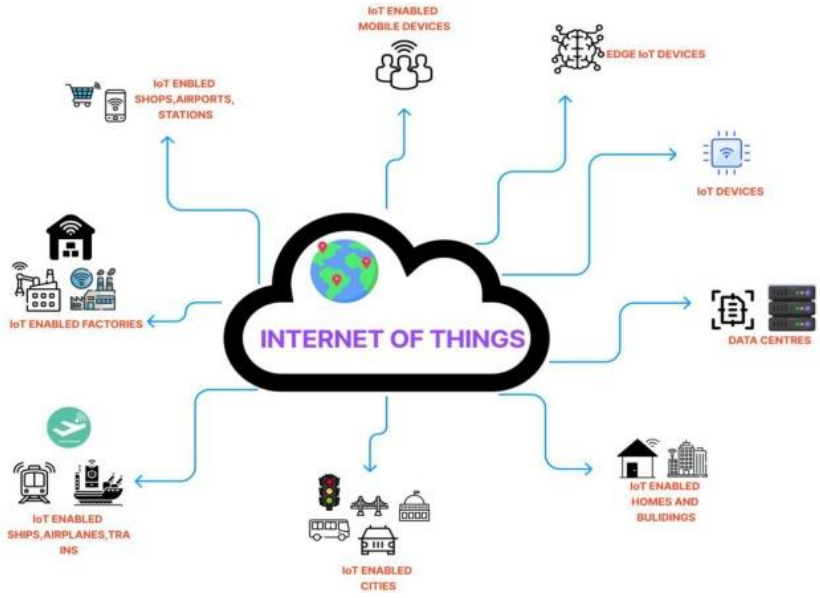


Fig. 1.1: Industrial IoT system.

To address these issues, intrusion detection technology has been deployed as an active network security measure. By continuously monitoring the network, it provides real-time protection against both internal and external intrusions. The characteristics of intrusion detection include proactiveness, real-timeliness, and dynamism, making it a promising research direction in the field of IoT security. Intrusion detection technology employs classifiers to distinguish between normal and abnormal data flows, generating alerts for potential attack behaviours.

The development of attack recognition technology has gone through three stages: pattern matching algorithms, machine learning algorithms and deep learning algorithms. The pattern matching algorithm was first applied to intrusion detection tasks based on feature matching. In

[1], Wu and Shen analyzed the classical pattern matching algorithms, BM and AC, and proposed the corresponding improved algorithms BMHS and AC-BM; experiments illustrated that the enhanced algorithms greatly optimize the timeliness of IDS.

Dagar et al. [2] applied RabinKarp and Knuth-MorrisPratt pattern matching algorithms to intrusion detection tasks and compared their execution efficiency. However, these pattern matching algorithms are difficult to adapt to today's network environment due to the diversity of network attacks. An attack recognition algorithm based on ML has been successfully applied to IDS and achieved excellent performance, which gradually replaced the traditional pattern matching algorithm. SVM is a typical supervised learning model in ML.

Thaseen and Kumar [3] presented a novel attack recognition model based on chi-square feature selection and multi class support vector machine (SVM). The simulation illustrates that removing redundant features significantly improves the calculation accuracy and execution efficiency of the model.

Ingre et al. [4] established a novel intrusion recognition system by combining the relevant feature screening algorithm with decision trees.

Problem Definition

In the contemporary digital landscape, the escalating threats of unauthorized access and suspicious activities within computer networks pose significant challenges to the security and integrity of sensitive data. The imperative to safeguard information traversing these networks has never been more crucial. Traditional machine learning algorithms, such as Naive Bayes and Support Vector Machines (SVM), have historically played a role in addressing these challenges. However, the dynamic and sophisticated nature of contemporary cyber threats necessitates a more nuanced and adaptable approach.

The central problem addressed by this research lies in the domain of wireless network intrusion detection. As the traditional algorithms exhibit limitations in handling the evolving threat landscape, there is a pressing need for an integrated system that combines the strengths of traditional methods with the adaptability of deep learning approaches. Enter the Evolving Cloud-based Neural Network (ECNN), a cutting-edge technique that holds the promise of discerning complex patterns and anomalies within network traffic.

Furthermore, the challenge extends beyond the technical intricacies of detection algorithms to the usability of such systems. Users without an in-depth understanding of machine learning or

programming face barriers in navigating and effectively utilizing intrusion detection systems. Therefore, the development of a user interface that simplifies the complex process of intrusion detection becomes a pivotal aspect of the problem statement.

The incorporation of innovative techniques like Adaptive Sampling and Synthesis (ASS) adds another layer of complexity to the problem. While traditional methods might struggle to keep pace with emerging threats, ASS aims to enhance the accuracy and efficiency of intrusion detection by introducing dynamic adaptability into the detection process.

In essence, the problem at hand is to create a comprehensive system that not only addresses the critical task of wireless network intrusion detection but also surmounts the challenges posed by the dynamic nature of contemporary cyber threats. This system must integrate traditional algorithms, deep learning approaches like ECNN, and innovative techniques like ASS, all while ensuring accessibility for users with varying levels of technical expertise. The overarching goal is to establish a robust and secure network environment capable of effectively detecting and mitigating unauthorized access and suspicious activities.

Research Motivation

The motivation for undertaking this research stems from the escalating threat landscape surrounding computer networks, where unauthorized access and suspicious activities pose a formidable risk to the security and integrity of sensitive data. In the contemporary digital era, where reliance on interconnected systems is pervasive, the need to develop an advanced intrusion detection system becomes imperative. The overarching goal is to fortify network security, particularly in the wireless domain, where vulnerabilities are pronounced and evolving.

The increasing sophistication of cyber threats underscores the urgency of devising a robust and adaptive solution capable of identifying unauthorized access and suspicious activities within computer networks. Traditional machine learning algorithms, such as Naive Bayes and Support Vector Machines (SVM), have been instrumental but are challenged by the dynamic nature of modern cyber threats. This limitation necessitates the integration of cutting-edge technologies like Evolving Cloud-based Neural Network (ECNN) to augment the system's ability to discern intricate patterns and anomalies within network traffic.

The motivation further extends to the imperative of democratizing access to intrusion detection capabilities. Recognizing that not all users possess an in-depth understanding of machine

learning or programming, the research seeks to bridge this knowledge gap by developing a user interface that simplifies the intricacies of the intrusion detection process. This user-friendly interface aims to empower a broader user base, enabling effective utilization of the system without the need for specialized technical expertise.

The incorporation of innovative techniques, such as Adaptive Sampling and Synthesis (ASS), into the research framework further amplifies the motivation. As cyber threats continually evolve, the research endeavors to enhance the accuracy and effectiveness of intrusion detection by introducing dynamic adaptability into the detection process. The goal is to create a system that not only meets the current security challenges but also anticipates and mitigates emerging threats proactively.

In essence, the motivation behind this research lies in the imperative to address the pressing and evolving cybersecurity challenges faced by computer networks. By integrating traditional algorithms with state-of-the-art technologies like ECNN and user-friendly interfaces, the research seeks to contribute to the creation of a comprehensive and accessible intrusion detection system. The overarching aim is to ensure a robust and secure network environment capable of effectively safeguarding sensitive data against unauthorized access and suspicious activities.

Significance

The significance of this research lies in its ability to enhance cybersecurity threat detection by leveraging machine learning and deep learning techniques, specifically ADASYN-based data balancing and SPC-CNN classification. Traditional security models struggle with imbalanced datasets, feature dependency, and low accuracy in detecting complex attacks (R2L and U2R). By incorporating ADASYN, the model effectively addresses data imbalance, ensuring fair representation of all attack types, while SPC-CNN automates feature extraction and improves classification accuracy. This research provides a scalable, high-performance solution for real-time network security, offering better precision, adaptability to evolving cyber threats, and reduced reliance on manual feature engineering, making it a crucial advancement in intelligent threat detection systems.

Applications

The applications of the proposed system for detecting unauthorized access and suspicious activities within a computer network are diverse and hold significant implications for enhancing the security and integrity of sensitive data in various contexts.

Corporate Security and Data Protection: In corporate settings, the developed intrusion detection system can be deployed to safeguard proprietary information, trade secrets, and confidential data. It acts as a vigilant guardian against unauthorized access attempts and potentially malicious activities, ensuring that the organization's sensitive data remains secure.

Financial Institutions: Banking and financial institutions handle vast amounts of sensitive data, including customer information and financial transactions. The intrusion detection system can play a pivotal role in preventing unauthorized access and fraudulent activities, contributing to the overall security posture of financial systems.

Healthcare Systems: In the healthcare sector, where patient privacy and confidentiality are paramount, the developed system can be instrumental in protecting electronic health records and sensitive medical information. Unauthorized access attempts can be swiftly identified and mitigated, ensuring compliance with privacy regulations.

Government Networks: Government agencies often deal with classified information and critical infrastructure data. The intrusion detection system can bolster the security of government networks, thwarting potential cyber threats and unauthorized access attempts that could compromise national security.

Educational Institutions: Universities and research institutions manage a wealth of research data, student records, and intellectual property. The developed system can contribute to maintaining the confidentiality and integrity of this information by identifying and preventing unauthorized access or suspicious activities.

Cloud Service Providers: With the increasing reliance on cloud services, providers can integrate the intrusion detection system to enhance the security of their platforms. This ensures the protection of client data stored in the cloud, offering a layer of defense against potential breaches.

Critical Infrastructure Protection: Systems controlling critical infrastructure, such as energy grids and transportation networks, are prime targets for cyber threats. The intrusion detection

system can play a crucial role in identifying and neutralizing potential attacks, safeguarding the continuity of essential services.

Small and Medium-sized Enterprises (SMEs): SMEs, which may lack extensive cybersecurity resources, can benefit from an accessible intrusion detection system. The user-friendly interface ensures that businesses without specialized technical expertise can still implement robust security measures.

Network Service Providers: Companies offering network services can deploy the intrusion detection system to monitor and secure their networks, ensuring the reliability and trustworthiness of their services.

In conclusion, the developed system holds broad applications across various sectors, contributing to the overarching goal of fortifying network security, preserving data integrity, and protecting sensitive information from unauthorized access and malicious activities.

1.1 EXISTING SYSTEM

Signature-Based Intrusion Detection Systems (IDS)

Signature-based IDS is one of the most traditional approaches to detecting cyber threats. It relies on a database of known attack signatures, which are predefined patterns of malicious activities. When network traffic or system logs match any of these signatures, the IDS flags the activity as a potential threat. These systems are widely used due to their high accuracy in detecting known threats and their relatively low false positive rates.

However, the primary limitation of signature-based IDS is its inability to detect zero-day attacks or novel security threats. Since it depends on predefined attack signatures, any new type of cyberattack that does not have a recorded signature remains undetected. Furthermore, keeping the signature database updated requires constant monitoring and maintenance, making it a resource-intensive process.

Another major drawback is that sophisticated attackers can evade detection by slightly modifying their attack patterns. Since the system matches exact patterns, even small variations in an attack can bypass detection. This makes signature-based IDS ineffective against polymorphic malware, evolving attack techniques, and adversaries that deliberately modify their attack behavior to avoid being flagged.

Rule-Based Security Systems

Rule-based security systems function by implementing a set of predefined security rules to monitor network behavior and detect anomalies. These rules are typically written by security analysts based on past attack patterns and best practices. The system continuously compares incoming traffic against these rules to identify suspicious activity, such as multiple failed login attempts or unusual data transfers.

Despite being useful in structured environments, rule-based security systems suffer from several limitations. First, they require constant manual updates as new threats emerge, which is time-consuming and demands expert knowledge. Additionally, rigid rule structures often fail to adapt to new attack techniques, leading to missed detections for evolving threats.

Another significant challenge is the high rate of false positives. Since these systems rely on fixed thresholds and conditions, benign activities may sometimes be flagged as security

breaches. This creates unnecessary alerts, increasing the workload of security teams and potentially causing important threats to be overlooked due to alert fatigue.

Log-Based Anomaly Detection Systems

Log-based security systems analyze system logs, network activity records, and user behavior logs to detect potential threats. These systems rely on manually set parameters to identify deviations from normal behavior, such as unauthorized access attempts, suspicious file modifications, or unusual network traffic spikes.

One of the major issues with log-based detection is the sheer volume of data that needs to be processed. Large enterprises generate massive amounts of log files daily, making manual analysis impractical. Security teams must sift through thousands of logs to find anomalies, increasing the chances of missing critical threats due to human oversight.

Moreover, these systems often struggle with false negatives, as they rely on historical patterns to detect threats. If an attacker operates in a way that does not deviate significantly from normal behavior, the system may fail to recognize the attack. Additionally, attackers can manipulate log files, either by deleting or altering records, making it difficult to trace malicious activities effectively.

Limitations

1. **Inability to Detect New Threats** – Most manual systems rely on predefined rules or signatures, making them ineffective against zero-day attacks and evolving threats.
2. **High False Positive and False Negative Rates** – Rigid thresholds and rule-based mechanisms often flag normal activities as attacks or fail to detect subtle malicious activities.
3. **Resource and Time Intensive** – Manual updating of rules, reviewing logs, and maintaining signature databases require significant human effort and time, reducing efficiency.
4. **Scalability Issues** – As network traffic and cyber threats grow exponentially, manual systems become inefficient in handling large-scale data, leading to performance bottlenecks.

5. **Evasion by Attackers** – Advanced attackers can modify attack patterns, manipulate logs, or employ obfuscation techniques to bypass detection, rendering manual systems ineffective.

1.2 PROPOSED SYSTEM

Overview

This novel security threat detection method has not been presented in existing surveys and effectively overcomes the drawbacks of traditional approaches by integrating preprocessing, data balancing (ADASYN), train-test splitting, and deep learning (CNN). Traditional security detection methods, such as Naïve Bayes and Support Vector Machine (SVM), struggle with complex attack patterns and imbalanced datasets, leading to low detection rates for minority attacks like R2L and U2R. To overcome these issues, we propose a CNN-based approach with ADASYN for security threat detection. Our methodology first preprocesses the dataset, ensuring high-quality input data. Next, ADASYN (Adaptive Synthetic Sampling) is applied to generate synthetic minority class samples, addressing class imbalance. The dataset is then split into training and testing sets to evaluate model performance. Unlike traditional classifiers, our Convolutional Neural Network (CNN) learns deep hierarchical features from network traffic, significantly improving classification accuracy. This approach enhances real-time detection, minimizes false positives, and ensures high precision, making it a superior alternative to existing machine learning-based security systems.

Step-1.:Data Preprocessing and Normalization

The first step involves cleaning and preprocessing the Security Threat Dataset, which consists of four attack labels: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). This includes:

- Removing duplicate, irrelevant, and inconsistent records to ensure data quality.
- Handling missing values using statistical imputation techniques.
- Normalizing numerical features to a 0-1 scale using Min-Max scaling, preventing bias in model training.
- Feature selection and dimensionality reduction (if necessary) to remove redundant features and improve model efficiency.

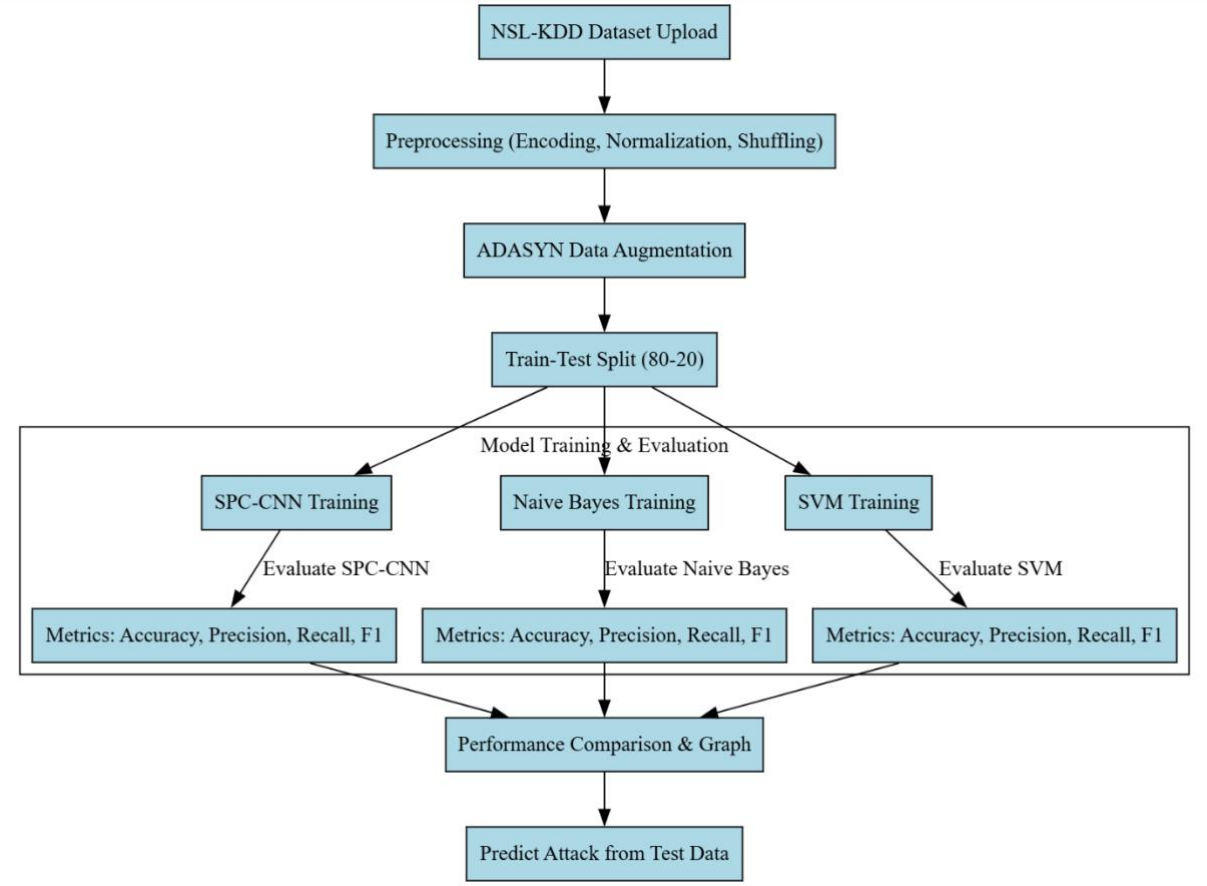


Fig. 4.1: Proposed block diagram of Security threat detection.

Step-2: Addressing Class Imbalance Using ADASYN

Security datasets often suffer from severe class imbalance, where minority attack types (e.g., R2L, U2R) have significantly fewer samples than majority classes (e.g., DoS). To address this, we use Adaptive Synthetic Sampling (ADASYN):

- ADASYN generates synthetic samples for minority attack classes, ensuring better balance.
- Unlike traditional oversampling methods, ADASYN prioritizes harder-to-classify samples, improving model learning.
- This step prevents the model from being biased toward majority classes and enhances detection performance for rare attacks.

Step-3: Train-Test Splitting for Model Evaluation

The dataset is split into training and testing sets using an 80-20 ratio, ensuring that the model is evaluated on unseen data. This helps assess its real-world performance and generalization capabilities.

Step-4. Proposed Deep Learning Model: Convolutional Neural Network (CNN)

To overcome the limitations of traditional machine learning models (Naïve Bayes, SVM), we introduce a CNN-based deep learning model for security threat detection. Unlike conventional approaches, CNN automatically learns hierarchical features from network traffic, significantly improving classification performance.

- Input Layer: Takes preprocessed network traffic data as input.
- Convolutional Layers: Extract spatial and temporal features from network data, enabling deeper pattern recognition.
- Pooling Layers: Reduce dimensionality while preserving key attack characteristics.
- Fully Connected Layers: Combine extracted features to classify network traffic into DoS, Probe, R2L, or U2R attacks.
- Softmax Activation: Outputs probability scores for each class, enabling accurate attack classification.

Step-5: Performance Evaluation

The model is evaluated using **key classification metrics**:

- **Accuracy**: Measures the overall correctness of the model.
- **Precision & Recall**: Assesses the model's ability to correctly detect attacks while minimizing false positives.
- **F1-Score**: Provides a balance between precision and recall, ensuring robust classification across all attack types.

Step-7: Comparative Analysis and Real-Time Deployment

- The CNN model is compared against traditional classifiers (Naïve Bayes, SVM), demonstrating superior accuracy and recall, especially for minority classes (R2L, U2R).

- The final trained CNN model is integrated into an Intrusion Detection System (IDS) for real-time security monitoring and automated threat detection.

By integrating preprocessing, ADASYN-based data balancing, train-test splitting, and deep learning (CNN), this method outperforms existing approaches in security threat detection and provides a robust, scalable, and highly accurate solution.

Data balancing with Adasyn(Adaptive Synthesis)

In machine learning, particularly in cybersecurity threat detection, datasets are often highly imbalanced. This means that some attack classes (e.g., DoS) may have thousands of records, while rarer attacks like User-to-Root (U2R) or Remote-to-Local (R2L) may have only a few instances. Class imbalance poses a major challenge because machine learning models tend to be biased toward majority classes, leading to poor detection rates for minority attack types. Standard classification algorithms may ignore rare attacks since they contribute less to the training error, resulting in an increased rate of false negatives.

To address this, oversampling techniques are used to artificially increase the number of samples in the minority classes. The given code applies ADASYN (Adaptive Synthetic Sampling) to balance the dataset by generating synthetic data points for underrepresented attack classes, making the dataset more uniform and improving model performance.

Step-1:Defining the ADASYN Object

- The code creates an object sm = SMOTE(random_state=2), which is typically used for oversampling, but it should be replaced with ADASYN if adaptive sampling is explicitly needed.
- The fit_sample(X, Y) function is then called to apply synthetic data generation to the original dataset (X, Y).

Step-2:Generating Synthetic Samples

- ADASYN first computes the density distribution of minority classes.
- It generates synthetic samples for the most difficult-to-learn instances rather than randomly oversampling all minority class samples equally.
- Unlike traditional SMOTE, ADASYN focuses only on minority class points that are difficult to classify, making the augmentation process more effective.

Step-3:Updating the Dataset

- After applying ADASYN, the dataset now contains a more balanced distribution of attack classes.
- The code then calculates the number of records in each class after augmentation using np.unique(Y, return_counts=True).
- The output is displayed in text format, showing how many records exist for each attack type after augmentation.

Step-4:Visualization of Data Balancing with ADASYN

Once the data is balanced, it is crucial to visually confirm the distribution of attack classes. The code:

- Creates a bar chart to display the count of each attack type after augmentation.
- Uses attack labels on the x-axis and the number of records on the y-axis to show the new distribution.
- Updates the title of the plot to reflect the dataset after augmentation.
- Displays the plot using plt.show(), providing a clear view of how ADASYN has adjusted the class distributions.

Train-test Splitting

Train-test splitting is a crucial step in building a robust security threat detection model. By allocating 80% of the data for training and 20% for testing, we ensure that the model has enough data to learn while still allowing us to measure its real-world effectiveness. A proper split prevents overfitting and underfitting, ensuring that the model generalizes well to new attack patterns. This step significantly improves the accuracy and reliability of cybersecurity threat detection systems.

Step 1: Define Train-Test Ratio

- The dataset is divided into 80% training data and 20% testing data to ensure the model learns from a larger portion while keeping enough for evaluation.

Step 2: Randomly Split the Data

- The dataset is randomly shuffled and split to avoid bias, ensuring all security threat categories (DoS, Probe, R2L, U2R) are fairly represented in both sets.

Step 3: Verify Data Distribution

- The number of records in both training and testing sets is checked to confirm that the split is properly balanced.

Step 4: Train and Test the Model

- The training set is used to teach the model attack patterns.
- The testing set evaluates how well the model detects unseen threats, ensuring reliable performance.

ML Model Building

Naïve Bayes

Naïve Bayes is a probabilistic classification algorithm based on Bayes' theorem, widely used in cybersecurity threat detection due to its speed and efficiency. It assumes that all features are independent, meaning that each feature contributes separately to the probability of an attack occurring. In this approach, the model is first trained on labeled network traffic data (X_train, y_train), learning the likelihood of different attack types based on past patterns. When new data (X_test) is provided, Naïve Bayes calculates the probability of each security threat type (DoS, Probe, R2L, U2R) and assigns the most likely attack label as the predicted output. The model's performance is then evaluated by comparing its predictions to the actual attack labels (y_test)shown in Fig-4.2. While Naïve Bayes is fast and effective for large datasets, it has limitations, such as assuming feature independence and struggling with complex, highly correlated attack patterns. Despite this, it remains a useful tool in cyber threat classification, especially when dealing with structured and well-preprocessed data.

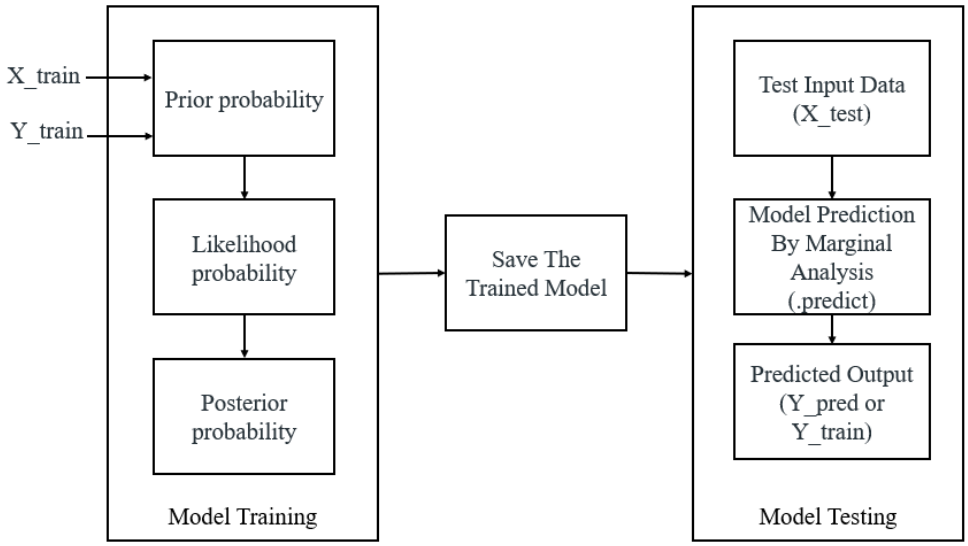


Fig.4.2 : Naïve Bayes Classifier

Step 1: Training the Naïve Bayes Model (Using X_train and y_train)

Naïve Bayes is a probabilistic classification algorithm that works by assuming that features in the dataset are independent. In this case, the training dataset (X_train, y_train) consists of network activity features labeled with attack types (DoS, Probe, R2L, U2R).

- The model first learns from X_train, which contains network traffic characteristics such as packet rates, connection durations, and protocol types.
- The corresponding labels in y_train indicate whether the activity is normal or belongs to a specific attack type.
- During training, Naïve Bayes calculates the probability of each attack type occurring given the observed features.

Since Naïve Bayes is fast and efficient, it can quickly process large security datasets, making it useful for real-time threat detection.

Step 2: Applying the Model to Test Data (Using X_test as Input)

Once trained, the model is tested on new, unseen data (X_test) to check its performance. X_test consists of network traffic features from real-time or historical logs but without attack labels.

- The trained model applies its probability-based rules to classify each record in X_test.
- It predicts whether each connection is normal or belongs to one of the attack categories.

- Since Naïve Bayes assumes independence among features, it calculates the likelihood of a threat based on individual attributes and combines them to determine the final prediction.

This step ensures that the model is able to recognize cyber threats based on patterns learned from the training phase.

Step 3: Predicting Attack Categories (Generating y_test Predictions)

After analyzing X_test, the model generates predicted outputs (y_test), which represent the detected attack types for each test record.

- Each predicted label in y_test is compared against the actual label to measure how accurately the model classifies security threats.
- If the predicted labels match the true attack types, the model is considered effective.
- If the predictions are incorrect, it may indicate that the model requires further improvements, such as feature selection or data balancing.

By using Naïve Bayes, security systems can quickly classify network traffic into normal and different attack types, making it a valuable tool for cybersecurity threat detection.

Limitations of Naïve Bayes

Feature Independence Assumption – Naïve Bayes assumes all features are independent, which is unrealistic in security threat datasets where network behaviors are interdependent.

- **Struggles with Complex Attacks** – It may perform poorly on advanced attacks like U2R and R2L, where attack patterns are subtle and feature correlations matter.
- **Highly Sensitive to Data Imbalance** – If certain attack types are underrepresented in training data, Naïve Bayes may fail to classify them correctly.
- **Limited Handling of Continuous Data** – While it can handle categorical data well, it may not work effectively with continuous numerical values without proper preprocessing.
- **Assumes Equal Importance of Features** – It treats all features as equally important, which may lead to incorrect classifications if some security features carry more weight than others.

CNN

Convolutional Neural Networks (CNNs) are a deep learning architecture designed to process and analyze complex patterns in data, particularly effective for image recognition and cybersecurity threat detection.

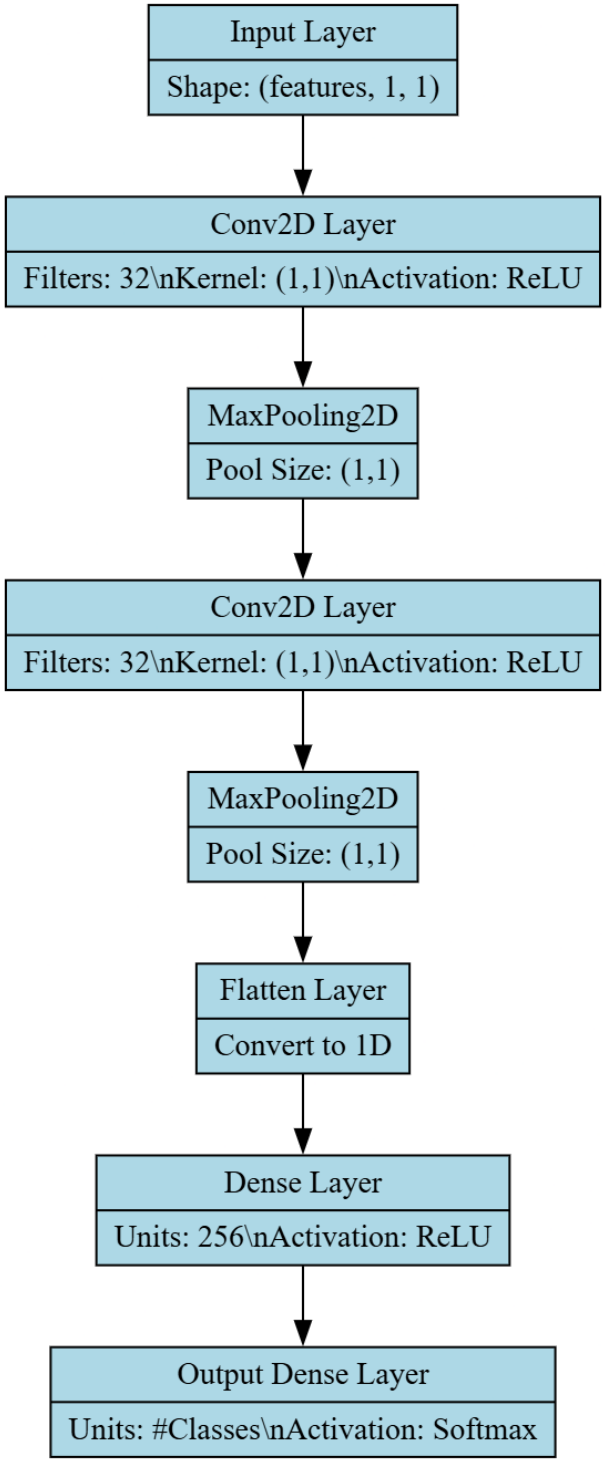


Fig. 4.3: Layered architecture of proposed deep learning model.

CNNs work by automatically extracting important features through layers of convolutional filters, which scan the input data (such as network traffic patterns) to detect meaningful relationships. These extracted features are then passed through pooling layers, which remove redundant and irrelevant information while preserving the most significant details. The processed features are then flattened and fed into fully connected layers, where the final classification is made using an activation function like softmax. In the context of security threat detection, CNNs can analyze network traffic, detect anomalies, and classify different types of attacks (DoS, Probe, R2L, U2R) with high accuracy. Their ability to automatically learn features makes them superior to traditional models that require manual feature selection, enabling real-time cybersecurity applications with improved precision and scalability.

Step 1: Understanding SPC-CNN for Security Threat Detection

SPC-CNN (Specialized Convolutional Neural Network) is a deep learning-based classification model designed to detect and classify security threats in network traffic. Unlike traditional models like Naïve Bayes or SVM, SPC-CNN can automatically extract and learn important features from raw data. It processes input data in multiple layers, capturing intricate patterns that distinguish between attack types such as DoS, Probe, R2L, and U2R. In this approach, the model is trained using X_train (features) and y_train (labels) and later tested on X_test to predict attack types, which are then evaluated against y_test.

Step 2: Data Preprocessing and Input Reshaping

Before training, the input data needs to be reshaped to fit the CNN model.

- X_train (training features) and X_test (testing features) are reshaped into a four-dimensional format to match the input structure required by Convolutional layers.
- The attack labels y_train and y_test are converted into categorical format using one-hot encoding to represent attack classes in a way suitable for classification.

This transformation allows CNN layers to process and analyze spatial patterns effectively.

Step 3: Convolutional Layers for Feature Extraction

- The first convolutional layer applies 32 filters to detect important features from network traffic data.

- These extracted features are passed through a MaxPooling layer, which removes redundant and irrelevant information while retaining only the most important features.
- A second convolutional layer further refines the feature extraction, followed by another MaxPooling layer to optimize the feature selection process.

This hierarchical feature extraction process ensures that only relevant network characteristics are used for classification, improving detection accuracy.

Step 4: Flattening and Fully Connected Layers

- After extracting features, the Flatten layer converts the extracted patterns into a 2D feature vector that can be used for classification.
- A Dense layer with 256 neurons applies deeper learning to the extracted features, improving classification performance.
- The final Dense output layer uses a softmax activation function to classify network traffic into four attack categories: DoS, Probe, R2L, and U2R.

These fully connected layers allow the CNN to make final predictions based on learned attack patterns.

Step 5: Model Training and Evaluation

- The model is compiled using the Adam optimizer and trained using categorical cross-entropy loss to minimize classification errors.
- If a pre-trained model does not exist, the model is trained for 20 epochs, saving the best model weights to improve performance.
- Once trained, the model predicts attack labels for X_test and compares them against y_test to evaluate accuracy, precision, recall, and F1-score.

SPC-CNN effectively classifies security threats by learning from raw network traffic data, making it highly adaptable for real-time attack detection.

Advantages of CNN

- **Automatic Feature Extraction** – Unlike traditional models, SPC-CNN learns important features directly from raw network data, reducing the need for manual feature selection.
- **High Accuracy and Performance** – Due to its deep learning capabilities, SPC-CNN achieves better accuracy than traditional machine learning models like Naïve Bayes or SVM.
- **Effective Handling of Complex Attack Patterns** – CNN layers can detect intricate attack behaviors that rule-based models struggle with, making it effective against advanced threats like R2L and U2R.
- **Optimized Feature Selection through MaxPooling** – MaxPooling layers remove redundant and irrelevant features, ensuring only the most significant information is used for classification.
- **Scalability for Large-Scale Network Security** – SPC-CNN can efficiently process large amounts of security data, making it suitable for real-time cybersecurity applications.

FUNCTIONAL REQUIREMENTS

Here is a list of functional requirements for the proposed system:

1. **User Interface Requirements:**
    - **GUI Dashboard:**
        - The system shall provide a graphical user interface (GUI) that displays a title, instructions, and a text area for logs and outputs.
        - The GUI shall include buttons for each major process (e.g., dataset upload, preprocessing, augmentation, training, testing, and prediction).
2. **Dataset Management:**
    - **Dataset Upload:**
        - The system shall allow users to select and upload the NSL-KDD dataset (CSV format) through a file dialog.
        - Upon upload, the system shall display the dataset's content and provide a summary (e.g., a bar graph showing attack distribution).
3. **Data Preprocessing:**
    - **Data Cleaning and Encoding:**
        - The system shall handle missing values by replacing them with default values (e.g., zero).
        - The system shall encode categorical variables (e.g., protocol_type, service, flag, label) using label encoding.
    - **Data Shuffling:**
        - The system shall randomize the dataset order to avoid bias during training.

4. **Data Augmentation:**
    - **SMOTE/ADASYN Augmentation:**
        - The system shall balance the dataset classes using SMOTE (Synthetic Minority Over-sampling Technique).
        - The system shall visualize the distribution of classes after augmentation using a bar chart.
5. **Data Splitting:**
    - **Training/Test Split:**
        - The system shall split the preprocessed dataset into training (80%) and testing (20%) subsets.
        - The system shall display details about the number of records in each subset.
6. **Model Training and Evaluation:**
    - **SPC-CNN Model:**
        - The system shall build a custom CNN (SPC-CNN) with layers such as Convolution2D, MaxPooling2D, Flatten, and Dense.
        - The system shall support saving and loading model weights.
        - The system shall train the CNN model on the training data and evaluate it on the test data.
    - **Traditional Machine Learning Models:**
        - The system shall support training and evaluation of a Gaussian Naive Bayes classifier.
        - The system shall support training and evaluation of an SVM classifier.

- **Performance Metrics:**
    - The system shall compute and display performance metrics including accuracy, precision, recall, and F1-score.
    - The system shall generate confusion matrices for each model.

7. **Visualization and Reporting:**
    - **Graphical Performance Comparison:**
        - The system shall generate an HTML table summarizing the performance metrics of the different algorithms.
        - The system shall create bar graphs comparing metrics across models.
    - **Confusion Matrix Visualization:**
        - The system shall display confusion matrices using a heatmap for each model after evaluation.
8. **Prediction Functionality:**
    - **New Data Prediction:**
        - The system shall allow users to upload new test data.
        - The system shall preprocess the new data using the same preprocessing steps as the training data.
        - The system shall use the trained SPC-CNN model to predict the attack types for the new data.
        - The system shall display the predictions alongside the original test data records.

9. **Data Persistence and File Management:**
    - **Model and History Storage:**
        - The system shall save model weights and training history to a file (e.g., using pickle) for future use.
    - **Report Generation:**
        - The system shall generate and open an HTML file to display the performance comparison table in a web browser.

These functional requirements ensure that the project covers end-to-end processing—from data management to model training, evaluation, and visualization—while providing a user-friendly interface for interacting with the IDS system.

1.3 SOFTWARE REQUIREMENT

Python is a high-level, interpreted programming language known for its simplicity and readability, which makes it a popular choice for beginners as well as experienced developers. Key features of Python include its dynamic typing, automatic memory management, and a rich standard library that supports a wide range of applications from web development to data science and machine learning. Its object-oriented approach and support for multiple programming paradigms allow developers to write clear, maintainable code. Python's extensive ecosystem of third-party packages further enhances its capabilities, enabling rapid development and prototyping across diverse fields.

Installation

First, download the appropriate installer from the official Python website (https://www.python.org/downloads/release/python-376/). For Windows users, run the executable installer and ensure to check the "Add Python to PATH" option during installation; for macOS and Linux, follow the respective package installation commands or use a package manager like Homebrew or apt-get. After installation, verify the setup by running python --version or python3 --version in your terminal or command prompt, which should display "Python 3.7.6." This version-specific installation supports all major functionalities and libraries compatible with Python 3.7.6, making it an excellent foundation for developing robust applications in areas such as data analysis, machine learning, and GUI development.

Python Packages

The project requires a robust set of software libraries and tools that work together to build an integrated system for plant disease classification. Below is an explanation of the key software requirements and the packages used:

- **Python:** The project is implemented in Python, which is chosen for its extensive ecosystem of libraries and its strong support for data analysis, machine learning, and GUI development.
- **Tkinter:** Used to build the graphical user interface (GUI) of the application. It handles tasks such as user authentication, data upload, and displaying results, making the system accessible to both admins and end-users.

- **PIL (Pillow):** Utilized for image processing, particularly for handling background images and other graphical elements within the GUI, thereby enhancing the visual appeal of the application.
- **Matplotlib & Seaborn:** These libraries are employed for data visualization. Matplotlib is used for creating standard plots, while Seaborn adds an extra layer of sophistication for statistical visualizations such as bar plots, violin plots, histograms, scatter plots, strip plots, and correlation heat maps.
- **Pandas & NumPy:** Essential for data manipulation and analysis. Pandas is used to load, preprocess, and analyze the CSV dataset, while NumPy supports numerical operations and data handling, which are crucial for processing large volumes of IoT data.
- **Scikit-learn (sklearn):** Provides the machine learning framework used in the project. It includes tools for model training, evaluation, train-test splitting, and data preprocessing (like label encoding). Models such as Gaussian Naive Bayes, SVM, KNN, and Decision Tree Classifier are implemented using scikit-learn.
- **Imbalanced-learn (imblearn):** Specifically used for implementing the SMOTE (Synthetic Minority Oversampling Technique) algorithm, which helps in addressing class imbalance in the dataset by generating synthetic samples for under-represented classes.
- **Joblib:** Utilized for saving and loading trained machine learning models. This ensures that once a model is trained, it can be stored and reused without retraining, thereby improving efficiency.
- **PyMySQL:** This package provides a means to connect to a MySQL database for handling user authentication. It facilitates operations such as user signup, login, and data storage, ensuring secure and persistent management of user credentials.

Each of these packages plays a crucial role in ensuring that the system is robust, scalable, and efficient—from data ingestion and preprocessing to model training, visualization, and deployment. The combination of these tools enables the creation of an integrated, user-friendly application for real-time plant disease classification and management.

1.4 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

- **Minimum Requirement**: 1 GHz single-core processor.
- **Recommended**: Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

- **Minimum Requirement**: 512 MB of RAM.
- **Recommended**: 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

- **Minimum Requirement**: 200 MB of free disk space for installation.
- **Recommended**: At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

- **Windows**: 32-bit and 64-bit systems, Windows 7 or later.
- **macOS**: macOS 10.9 or later.
- **Linux**: Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER 2

LITERATURE SURVEY

Nancy et al. [5] designed a dynamic recursive feature selection algorithm. By extending the decision tree algorithm and combining it with convolutional neural networks. They proposed an intelligent fuzzy temporal decision tree algorithm. The new algorithm achieved a high detection rate of unknown attacks on the KDD cup dataset.

An improved IDS based on a Bayesian network and feature selection algorithm was proposed in [6]. Although these ML detection algorithms achieved higher recognition accuracy in intrusion detection tasks, they not only need large-scale feature engineering but the model parameters are also difficult to adjust. However, the DL algorithm can autonomously abstract features from basic network traffic without complex feature engineering. Therefore, related research on intrusion detection is gradually focused on the DL method.An LSTM classifier with a gradient descent optimizer is used in IDS [7], which can effectively mine the association between features from the perspective of time.

Su et al. [8] combined an attention mechanism and BLSTM (bidirectional long short-term memory) to propose a network anomaly detection model BAT, which extracts coarse-grained features by connecting forward LSTM and backward LSTM. The BAT model uses an attention mechanism to filter the network flow vectors generated by the BLSTM model to obtain the key characteristics of network traffic classification.

Wei et al. [9] applied particle swarm optimization (PSO) to optimize the structure of DBN, and the improved DBN achieves significant anomaly detection ability.

Gao et al. [10] designed an effective attack recognition method by combining association rules and improved deep neural networks (DNN), which uses the apriori algorithm to mine the association between discrete features and labels to improve the recognition accuracy. Yin et al. [10] presented an effective attack recognition model by using feature enhancement and improved RNN; however, the feature enhancement method also increases the computational complexity of the model. CNN has been successfully applied to intrusion detection tasks because it can extract network traffic characteristics more effectively [4]. Lin et al. [5] designed a character-level CL-CNN model. The character-based encoding method makes the features more discretized, which contributes to improving the detection accuracy of IDS.

Wu et al. [11] designed a CNN model with a simple structure and proved the necessity of converting the original data into a 2D format through experiments. In addition, the combination of this simple CNN and 2D data conversion greatly improves the detection efficiency of the model compared with the RNN model in [12]. Ding and Zhai [5] proposed a convolutional neural network model (MS-CNN) based on multistage features. Multistage features are obtained by connecting the outputs of all convolutional layers to the dense layer with the softmax classifier. By adding supplementary information (such as local information and detailed information lost by higherlevel convolutional layers), the expressive ability of the model significantly improves.

Yang and Wang [13] extracted diverse features through a cross-layer aggregated CNN model, which greatly improved the expression ability of the model. Although the above attack recognition algorithms using CNN improve the detection accuracy, they ignore the interchannel information redundancy in the convolution layer. However, we cannot directly discard some channel information because we are not sure which channels are redundant.

To reasonably eliminate the problem of interchannel information redundancy, Zhang et al. [14] proposed a split-based plug and play convolution (SPC) block, which divides the channel of the convolution layer into two parts, the representative part and the uncertain redundant part, after which hierarchical processing is performed. Inspired by this idea, we propose an SPC-equipped CNN (SPC-CNN) and apply it to attack recognition tasks. The simulation illustrates that the comprehensive performance of SPC-CNN has obvious advantages compared with the traditional CNN model. Before that, the ADASYN data augmentation algorithm is used to prevent the model from being sensitive to large samples and ignores small samples. Then, the AS-CNN model mixed with the ADASYN algorithm and SPC-CNN is used for intrusion detection tasks. The simulation illustrates that the comprehensive performance of the hybrid AS-CNN model is better than that of using SPC-CNN alone.

CHAPTER 3

DESIGN OF THE MAJOR PROJECT

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process also be added to; or associated with, UML.

The Unified Modeling Language Is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS: The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an “is-a” or “has-a” relationship. Each class in the class diagram may be capable of providing certain

functionalities. These functionalities provided by the class are termed “methods” of the class. Apart from this, each class may have certain “attributes” that uniquely identify the class.

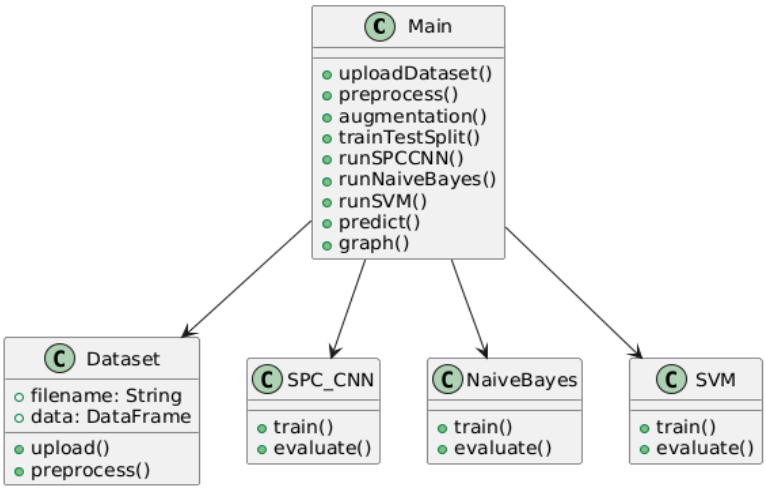


Fig. 5.1: Class diagram.

Data flow diagram

A Data Flow Diagram (DFD) is a visual representation of the flow of data within a system or process. It is a structured technique that focuses on how data moves through different processes and data stores within an organization or a system. DFDs are commonly used in system analysis and design to understand, document, and communicate data flow and processing.

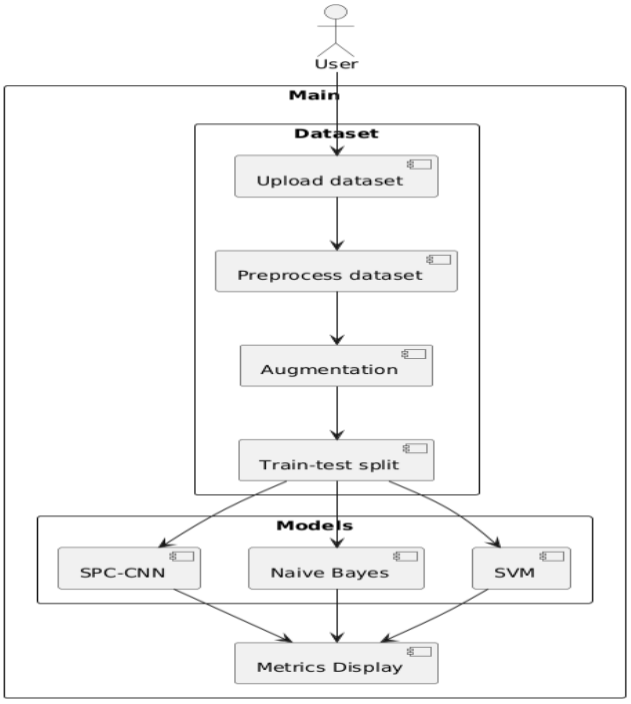


Fig. 5.2: Dataflow diagram.

Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows, as parallel vertical lines ("lifelines"), different processes or objects that live simultaneously, and as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

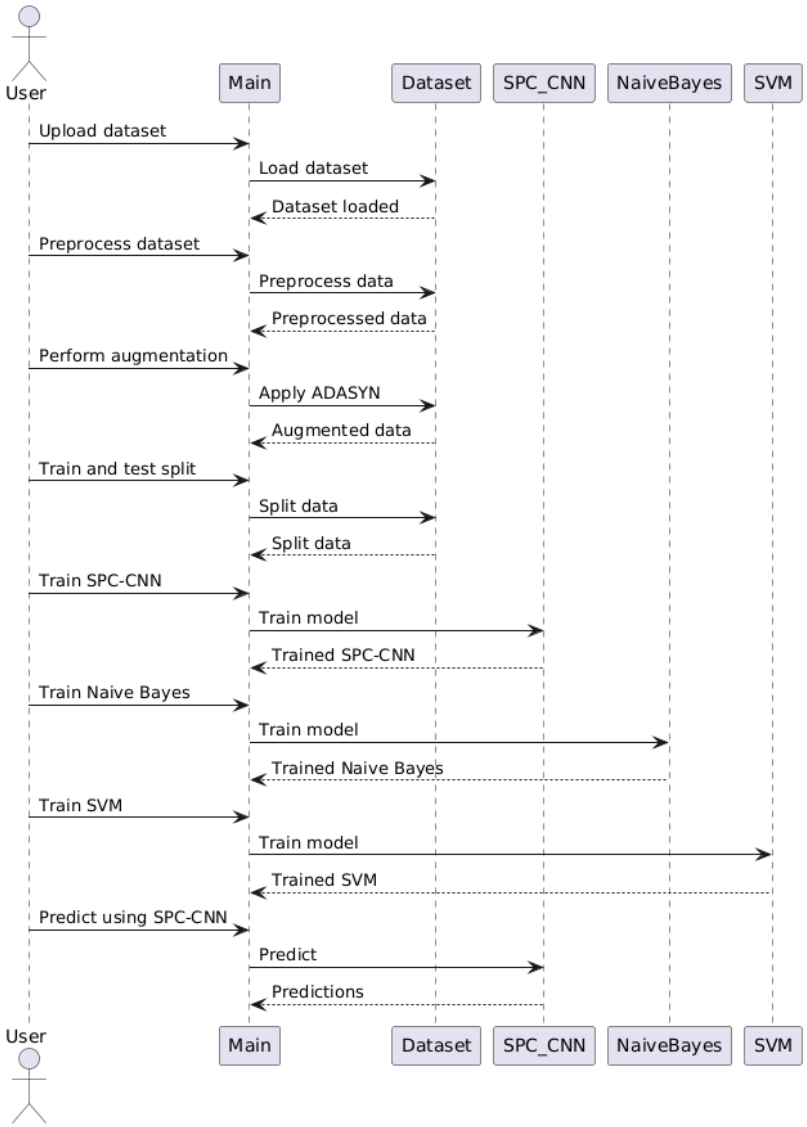


Fig. 5.3: Sequence diagram.

Activity diagram

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.

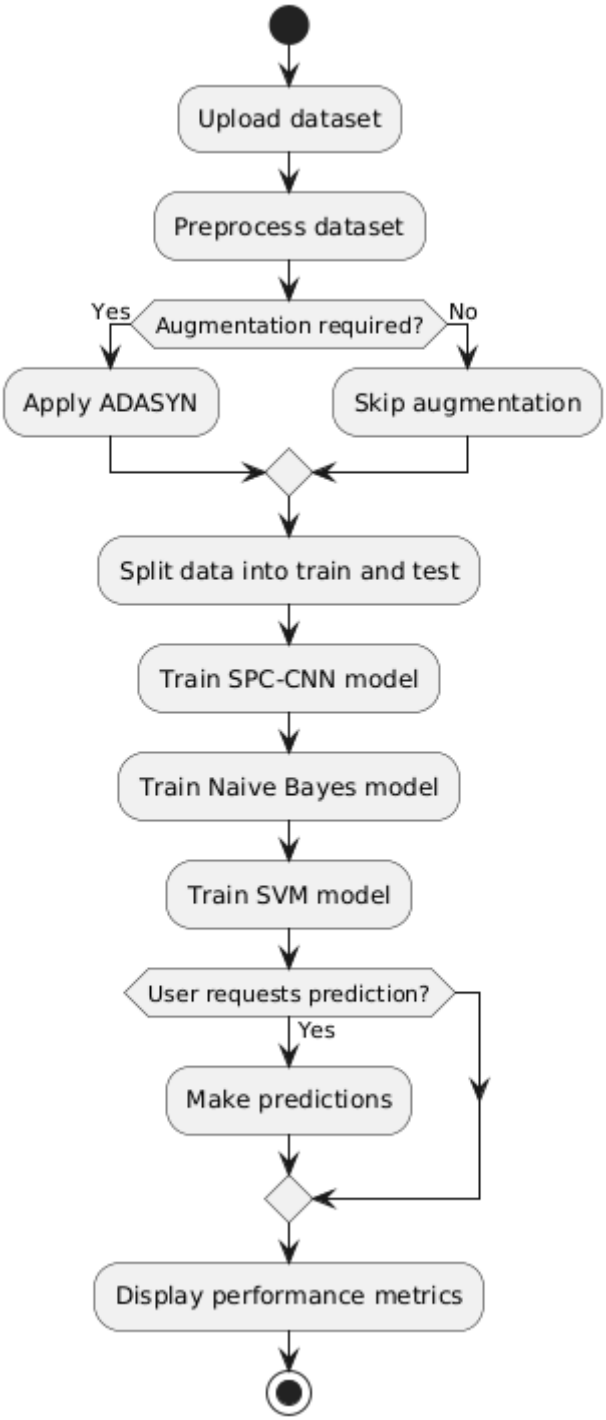


Fig. 5.4: Activity diagram.

Deployment diagram: The deployment diagram visualizes the physical hardware on which the software will be deployed.

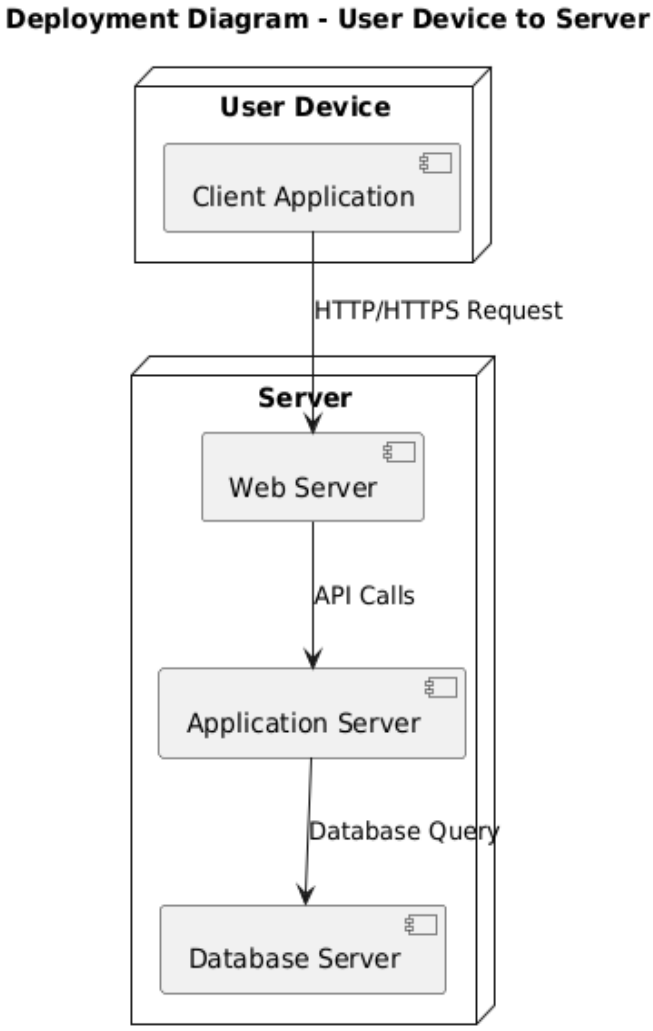


Fig. 5.5: Deployment diagram.

Use case diagram: The purpose of use case diagram is to capture the dynamic aspect of a system.

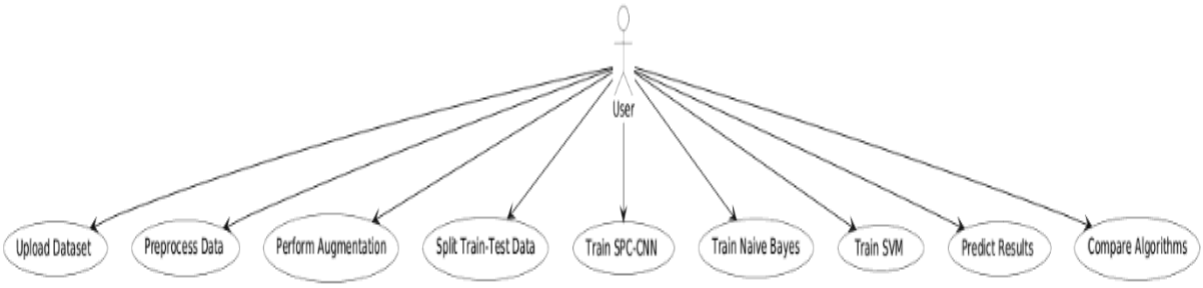


Fig. 5.6: Use case diagram.

Component diagram: Component diagram describes the organization and wiring of the physical components in a system.

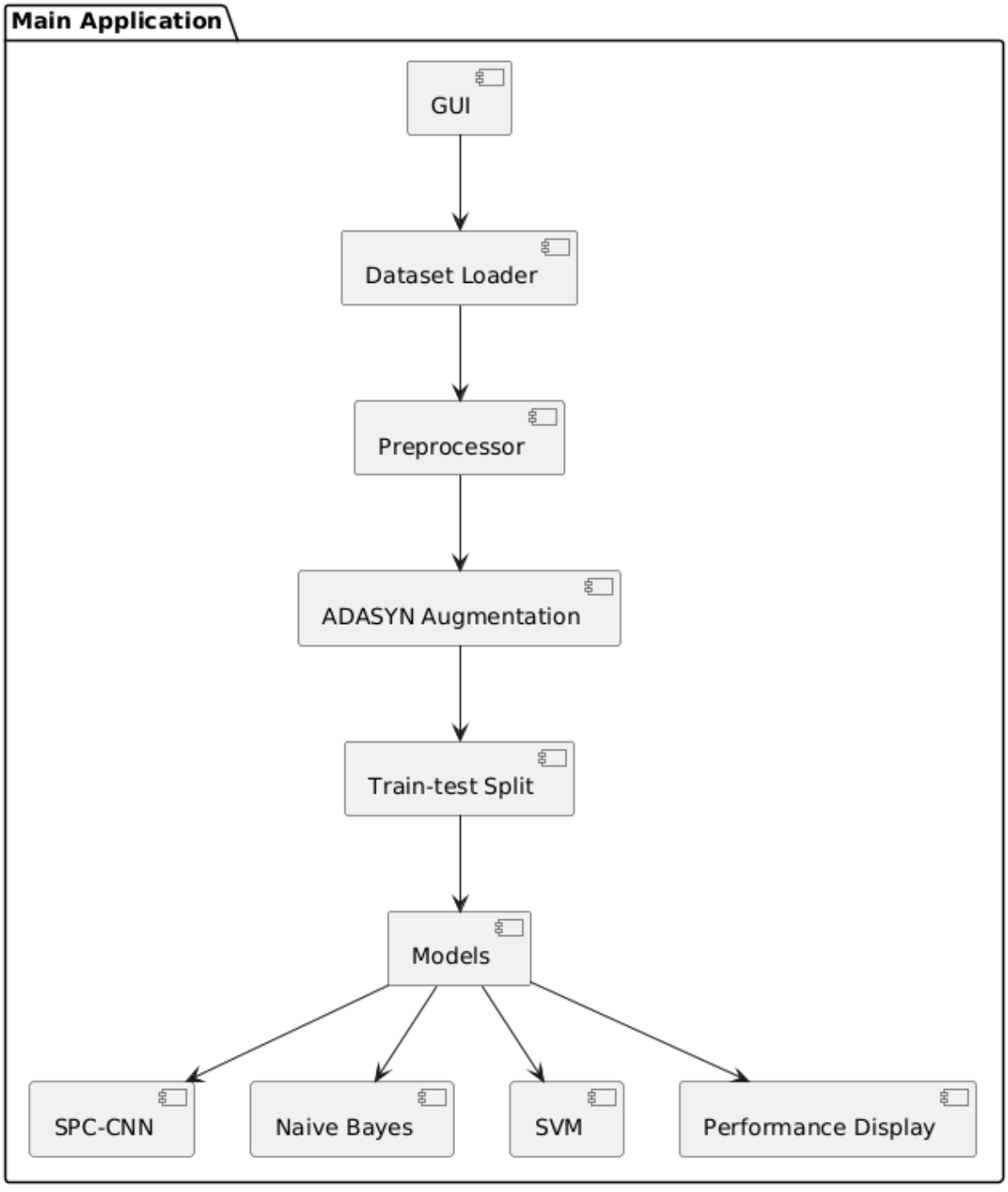


Fig. 5.7: Component diagram.

CHAPTER 4

IMPLEMENTATION

```
from tkinter import messagebox

from tkinter import *

from tkinter import simpledialog

import tkinter

from tkinter import filedialog

import matplotlib.pyplot as plt

from tkinter.filedialog import askopenfilename

import numpy as np

from keras.utils.np_utils import to_categorical

from keras.layers import  MaxPooling2D

from keras.layers import Dense, Dropout, Activation, Flatten

from keras.layers import Convolution2D

from keras.models import Sequential

import pickle

from sklearn.model_selection import train_test_split

import cv2

import os

from keras.callbacks import ModelCheckpoint

import webbrowser

from sklearn.metrics import confusion_matrix

from sklearn.metrics import accuracy_score

from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
import seaborn as sns
import pandas as pd
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE
from sklearn.naive_bayes import GaussianNB
from sklearn import svm
from sklearn.preprocessing import LabelEncoder

main = tkinter.Tk()
main.title("IIoT IDS")
main.geometry("1300x1200")

global filename, spc_cnn, X, Y, dataset
global X_train, X_test, y_train, y_test
global accuracy, precision, recall, fscore
global labels, scaler, le1, le2, le3, le4

def uploadDataset():
    global dataset, labels
    filename = filedialog.askopenfilename(initialdir="Dataset")
    text.delete('1.0', END)
    text.insert(END,filename+" loaded\n")
```

```
    dataset = pd.read_csv(filename)
    text.insert(END, str(dataset))
    labels = np.unique(dataset['label'])
    label = dataset.groupby('label').size()
    label.plot(kind="bar")
    plt.title("NSL-KDD Attack Graph")
    plt.xlabel("Attack Name")
    plt.ylabel("Count")
    plt.show()

def preprocess():
    global dataset, X, Y, le1, le2, le3, le4
    text.delete('1.0', END)
    cols = ['protocol_type','service','flag','label']
    le1 = LabelEncoder()
    le2 = LabelEncoder()
    le3 = LabelEncoder()
    le4 = LabelEncoder()
    dataset.fillna(0, inplace = True)
    dataset[cols[0]] = pd.Series(le1.fit_transform(dataset[cols[0]].astype(str)))
    dataset[cols[1]] = pd.Series(le2.fit_transform(dataset[cols[1]].astype(str)))
    dataset[cols[2]] = pd.Series(le3.fit_transform(dataset[cols[2]].astype(str)))
    dataset[cols[3]] = pd.Series(le4.fit_transform(dataset[cols[3]].astype(str)))
    data = dataset.values
```

```
    X = data[:,0:data.shape[1]-1]
    Y = data[:,data.shape[1]-1]
    print(X)
    print(Y)
    indices = np.arange(X.shape[0]) #shuffling the images
    np.random.shuffle(indices)
    X = X[indices]
    Y = Y[indices]
        text.insert(END,"Dataset preprocessing & normalization & shuffling process completed\n\n")
    text.insert(END,str(dataset.head()))


def augmentation():
    text.delete('1.0', END)
    global X, Y
    #now apply smote augmentation to balance data by adding synthesize records to fewer class
    sm = SMOTE(random_state = 2) #defining smote object
    X, Y = sm.fit_sample(X, Y)
    unique, count = np.unique(Y, return_counts=True)
    text.insert(END,"Records in each class after applying ADASYN Augmentation\n\n")
    text.insert(END,labels[0]+" Number of records : "+str(count[0])+"\n")
    text.insert(END,labels[1]+" Number of records : "+str(count[1])+"\n")
    text.insert(END,labels[2]+" Number of records : "+str(count[2])+"\n")
    text.insert(END,labels[3]+" Number of records : "+str(count[3])+"\n")
```

```
    height = count
    bars = labels
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.xticks(y_pos, bars)
    plt.title("NSL-KDD Attack Graph After ADASYN Augmentation")
    plt.xlabel("Attack Name")
    plt.ylabel("Count")
    plt.show()
    plt.show()


def trainTestSplit():
    text.delete('1.0', END)
    global X_train, X_test, y_train, y_test, X, Y
    X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train
and test
    text.insert(END,"Total records found in dataset : "+str(X.shape[0])+"\n\n")
    text.insert(END,"Dataset Train and Test Split\n\n")
    text.insert(END,"80% records used to train Algorithms : "+str(X_train.shape[0])+"\n")
    text.insert(END,"20% records used to test Algorithms : "+str(X_test.shape[0])+"\n")
    text.update_idletasks()



def calculateMetrics(algorithm, predict, y_test):
    a = accuracy_score(y_test,predict)*100
```

```
    p = precision_score(y_test, predict,average='macro') * 100
    r = recall_score(y_test, predict,average='macro') * 100
    f = f1_score(y_test, predict,average='macro') * 100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    text.insert(END,algorithm+" Accuracy  :  "+str(a)+"\n")
    text.insert(END,algorithm+" Precision : "+str(p)+"\n")
    text.insert(END,algorithm+" Recall    : "+str(r)+"\n")
    text.insert(END,algorithm+" FScore    : "+str(f)+"\n\n")
    text.update_idletasks()
    conf_matrix = confusion_matrix(y_test, predict)
    plt.figure(figsize =(6, 6))
     ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True,
cmap="viridis" ,fmt ="g");
    ax.set_ylim([0,len(labels)])
    plt.title(algorithm+" Confusion matrix")
    plt.ylabel('True class')
    plt.xlabel('Predicted class')
    plt.show()


def runSPCCNN():
    global X_train, X_test, y_train, y_test, spc_cnn
    global accuracy, precision, recall, fscore
```

```
text.delete('1.0', END)

accuracy = []

precision = []

recall = []

fscore = []

X_train1 = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1, 1))

X_test1 = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1, 1))

y_train1 = to_categorical(y_train)

y_test1 = to_categorical(y_test)

#defining CNN2D layers

spc_cnn = Sequential()

#cnn layer to filtered features and then send to max pooling layer

        spc_cnn.add(Convolution2D(32, (1, 1), input_shape = (X_train1.shape[1],X_train1.shape[2],X_train1.shape[3]), activation = 'relu'))

#max pooling layer remove irrrelavant and redundant fetaures and then select only optimized features to train SPC-CNN model

spc_cnn.add(MaxPooling2D(pool_size = (1, 1)))

spc_cnn.add(Convolution2D(32, (1, 1), activation = 'relu'))

spc_cnn.add(MaxPooling2D(pool_size = (1, 1)))

#flatten will be used to convert dataset features in to 2D format

spc_cnn.add(Flatten())

spc_cnn.add(Dense(output_dim = 256, activation = 'relu'))

spc_cnn.add(Dense(output_dim = y_train1.shape[1], activation = 'softmax'))

    spc_cnn.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy']) #compiling custome model
```

```
    if os.path.exists('model/model_weights.hdf5') == False:
        model_check_point = ModelCheckpoint(filepath='model/model_weights.hdf5', verbose = 1, save_best_only = True)
        #training the model
        hist = spc_cnn.fit(X_train1, y_train1, batch_size = 16, epochs = 20, validation_data=(X_test1, y_test1), callbacks=[model_check_point], verbose=1)
        f = open('model/history.pckl', 'wb')
        pickle.dump(hist.history, f)
        f.close()
    else:
        spc_cnn.load_weights('model/model_weights.hdf5')
    predict = spc_cnn.predict(X_test1)
    predict = np.argmax(predict, axis=1)
    target = np.argmax(y_test1, axis=1)
    calculateMetrics("Propose ADASYN-SPC-CNN", predict, target)


def runNaiveBayes():
    global X_train, X_test, y_train, y_test
    nb = GaussianNB()
    nb.fit(X_train, y_train)
    predict = nb.predict(X_test)
    calculateMetrics("Naive Bayes", predict, y_test)


def runSVM():
    global X_train, X_test, y_train, y_test
```

```
    svm_cls = svm.SVC()
    svm_cls.fit(X_train, y_train)
    predict = svm_cls.predict(X_test)
    calculateMetrics("SVM", predict, y_test)


def graph():
        output = "<html><body><table align=center border=1><tr><th>Algorithm Name</th><th>Accuracy</th><th>Precision</th><th>Recall</th>"
    output+="<th>FSCORE</th></tr>"
                                    output+="<tr><td>Propose ADASYN-SPC-CNN</td><td>"+str(accuracy[0])+"</td><td>"+str(precision[0])+"</td><td>"+str(recall[0])+"</td><td>"+str(fscore[0])+"</td></tr>"
                                                                output+="<tr><td>Naive Bayes</td><td>"+str(accuracy[1])+"</td><td>"+str(precision[1])+"</td><td>"+str(recall[1])+"</td><td>"+str(fscore[1])+"</td></tr>"
    output+="<tr><td>SVM</td><td>"+str(accuracy[2])+"</td><td>"+str(precision[2])+"</td><td>"+str(recall[2])+"</td><td>"+str(fscore[2])+"</td></tr>"
    output+="</table></body></html>"
    f = open("table.html", "w")
    f.write(output)
    f.close()
    webbrowser.open("table.html",new=2)


     df = pd.DataFrame([['Propose ADASYN-SPC-CNN','Precision',precision[0]],['Propose ADASYN-SPC-CNN','Recall',recall[0]],['Propose ADASYN-SPC-CNN','F1 Score',fscore[0]],['Propose ADASYN-SPC-CNN','Accuracy',accuracy[0]],
```

```
['Existing Naive Bayes','Precision',precision[1]],['Existing Naive Bayes','Recall',recall[1]],['Existing Naive Bayes','F1 Score',fscore[1]],['Existing Naive Bayes','Accuracy',accuracy[1]],

['SVM','Precision',precision[2]],['SVM','Recall',recall[2]],['SVM','F1 Score',fscore[2]],['SVM','Accuracy',accuracy[2]],

],columns=['Algorithms','Performance Output','Value'])

    df.pivot("Algorithms", "Performance Output", "Value").plot(kind='bar')

    plt.title("All Algorithms Performance Graph")

    plt.show()


def predict():

    global spc_cnn, scaler, le1, le2, le3, le4, labels

    cols = ['protocol_type','service','flag']

    text.delete('1.0', END)

    filename = filedialog.askopenfilename(initialdir="Dataset")

    dataset = pd.read_csv(filename)

    dataset.fillna(0, inplace = True)

    temp = dataset.values

    dataset[cols[0]] = pd.Series(le1.transform(dataset[cols[0]].astype(str)))

    dataset[cols[1]] = pd.Series(le2.transform(dataset[cols[1]].astype(str)))

    dataset[cols[2]] = pd.Series(le3.transform(dataset[cols[2]].astype(str)))

    dataset = dataset.values

    #dataset = scaler.transform(dataset)

    dataset = np.reshape(dataset, (dataset.shape[0], dataset.shape[1], 1, 1))

    predict = spc_cnn.predict(dataset)
```

```
    predict = np.argmax(predict, axis=1)
    print(predict)
    for i in range(len(predict)):
            text.insert(END,"Test Data = "+str(temp[i])+" Predicted As ====> "+labels[predict[i]]+"\n\n")


font = ('times', 15, 'bold')
title = Label(main, text='An Innovative Approach of Intrusion Combat Model for Industrial Internet of Things: An Application of Industry 4.0')
title.config(bg='firebrick4', fg='dodger blue')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=150)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=50,y=120)
text.config(font=font1)

font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload NSL-KDD Dataset", command=uploadDataset, bg='#ffb3fe')
```

```
uploadButton.place(x=50,y=550)

uploadButton.config(font=font1)


processButton = Button(main, text="Preprocess Dataset", command=preprocess, bg='#ffb3fe')

processButton.place(x=340,y=550)

processButton.config(font=font1)


adasynButton1 = Button(main, text="ADASYN Augmentation", command=augmentation, bg='#ffb3fe')

adasynButton1.place(x=570,y=550)

adasynButton1.config(font=font1)


splitButton = Button(main, text="Dataset Train & Test Split", command=trainTestSplit, bg='#ffb3fe')

splitButton.place(x=870,y=550)

splitButton.config(font=font1)


spccnnButton = Button(main, text="Train SPC-CNN Algorithm", command=runSPCCNN, bg='#ffb3fe')

spccnnButton.place(x=50,y=600)

spccnnButton.config(font=font1)


existingButton = Button(main, text="Run Naive Bayes Algorithms", command=runNaiveBayes, bg='#ffb3fe')

existingButton.place(x=340,y=600)
```

```
existingButton.config(font=font1)

graphButton = Button(main, text="Run SVM Algorithm", command=runSVM, bg='#ffb3fe')
graphButton.place(x=570,y=600)
graphButton.config(font=font1)

predictButton = Button(main, text="Comparison Graph", command=graph, bg='#ffb3fe')
predictButton.place(x=870,y=600)
predictButton.config(font=font1)

predictButton = Button(main, text="Predict Attack from Test Data", command=predict, bg='#ffb3fe')
predictButton.place(x=50,y=650)
predictButton.config(font=font1)

main.config(bg='LightSalmon3')
main.mainloop()
```

CHAPTER 5

RESULTS

Implementation description

This research implements IDS for Industrial Internet of Things (IIoT) environments using Machine Learning (ML) and Deep Learning (DL) techniques. The primary goal is to detect and classify network intrusions by analyzing network traffic data from the NSL-KDD dataset. The application provides a Graphical User Interface (GUI) using Tkinter, allowing users to interactively process data, train models, and visualize results.

1. **Dataset Handling and Attack Visualization**
    - The system allows users to upload the NSL-KDD dataset, a widely used benchmark dataset for intrusion detection in IoT and industrial networks.
    - The uploaded dataset is displayed in the Graphical User Interface (GUI).
    - A bar plot is generated to visualize the distribution of different attack types, helping in understanding the dataset's initial structure.
2. **Data Preprocessing for Industrial IoT Intrusion Detection**
    - Handling missing values: Any missing data is filled with appropriate values to avoid training issues.
    - Feature encoding: The categorical features such as protocol_type, service, and flag are converted into numerical representations using Label Encoding to make them suitable for machine learning models.
    - Shuffling & normalization: The dataset is shuffled to avoid biased learning and normalized to enhance training efficiency.
3. **Class Imbalance Handling using ADASYN Augmentation**
    - Since IIoT intrusion datasets are often highly imbalanced, the ADASYN (Adaptive Synthetic Sampling) technique is applied.
    - ADASYN helps generate synthetic data for underrepresented attack types, ensuring a balanced dataset that improves model performance.

- The post-augmentation class distribution is visualized using a bar graph, demonstrating the effectiveness of balancing the dataset.

4. **Train-Test Splitting for Effective Model Training**
    - The dataset is split into 80% training data and 20% testing data, ensuring an optimal balance between model learning and evaluation.
    - The GUI displays the exact number of records allocated for training and testing.
5. **Implementation of Intrusion Detection Models**
    - **Proposed SPC-CNN Model (ADASYN-SPC-CNN):**
        - A custom-built 2D Convolutional Neural Network (CNN) is implemented with multiple convolutional layers, max pooling layers, and dense layers to extract deep patterns from network traffic data.
        - The CNN model identifies attacks autonomously by learning hierarchical representations.
        - The model is trained using Adam optimizer and categorical cross-entropy loss function, with real-time model evaluation during training.
    - **Existing Machine Learning Models for Comparison:**
        - **Naïve Bayes Classifier (GNB):** A Gaussian Naïve Bayes model is implemented to act as a baseline classifier for intrusion detection.
        - **Support Vector Machine (SVM):** An SVM classifier is trained to compare its performance against deep learning models.
6. **Performance Evaluation Metrics for IIoT Intrusion Detection**
    - Each model is evaluated using the following metrics:
        - **Accuracy:** Measures the overall detection performance.
        - **Precision:** Evaluates how many of the detected intrusions were correctly classified.
        - **Recall:** Determines the ability of the model to detect all attack cases.

        - **F1-Score:** A balance between precision and recall, providing an overall performance measure.
    - The results are displayed in the GUI and plotted using Seaborn heatmaps and bar charts.

7. **Comparison and Graphical Analysis**
    - A comparison table is generated in HTML format and displayed in a web browser to compare the results of different models.
    - A bar chart is plotted to visually compare the performance of ADASYN-SPC-CNN, Naïve Bayes, and SVM on different evaluation metrics.
8. **Real-Time Attack Detection and Prediction**
    - Users can upload new test data to predict attacks using the trained SPC-CNN model.
    - The model classifies each test sample and displays the predicted attack type in the GUI.

Dataset Description

The dataset used in research is NSL-KDD dataset, which is widely used for security threat detection. Below is a detailed explanation of the columns in our dataset:

1. Basic Features (Network Traffic Attributes)

These attributes describe fundamental characteristics of network connections:

- **duration** – The length of the connection (in seconds).
- **protocol_type** – The protocol used for the connection (e.g., TCP, UDP, ICMP).
- **service** – The network service on the destination (e.g., HTTP, FTP, SSH).
- **flag** – The status flag of the connection, indicating whether it was successful or failed.
- **src_bytes** – The number of data bytes sent from the source to the destination.
- **dst_bytes** – The number of data bytes sent from the destination to the source.

2. Content Features (Intrusion-Specific Indicators)

These attributes analyze the payload of the packets to detect suspicious activity:

- **land** – A binary value (0 or 1) indicating whether the source and destination IPs and ports are the same (potential loop attack).
- **wrong_fragment** – The number of wrong or fragmented packets in the connection.
- **urgent** – The number of urgent packets sent, often used in DoS attacks.
- **hot** – The count of "hot" indicators such as login failures or execution of system commands.
- **num_failed_logins** – The number of failed login attempts (used to detect brute-force attacks).
- **logged_in** – A binary value indicating whether the login attempt was successful (1) or failed (0).
- **num_compromised** – The number of compromised conditions detected in the connection.
- **root_shell** – A binary indicator of whether root-level access was obtained.
- **su_attempted** – The number of times a su command (privilege escalation) was attempted.
- **num_root** – The number of root-level commands executed.
- **num_file_creations** – The number of file creation operations during the connection.
- **num_shells** – The number of shell sessions opened.
- **num_access_files** – The number of file access operations.
- **num_outbound_cmds** – The number of outbound commands in an FTP session.
- **is_host_login** – A binary indicator (0 or 1) specifying if the login attempt was made on the host machine.
- **is_guest_login** – A binary indicator specifying if the login was a guest account.

3. Time-Based Traffic Features (Connection Behavior Over Time)

These attributes capture connection activity over a specific window of time:

- **count** – The number of connections from the same source IP in a given timeframe.
- **srv_count** – The number of connections to the same service within a specific time window.
- **serror_rate** – The percentage of connections with SYN errors (possible DoS attack).
- **srv_serror_rate** – The percentage of connections to the same service with SYN errors.
- **rerror_rate** – The percentage of connections with REJ (reject) errors.
- **srv_rerror_rate** – The percentage of connections to the same service with REJ errors.
- **same_srv_rate** – The percentage of connections to the same service.
- **diff_srv_rate** – The percentage of connections to different services.
- **srv_diff_host_rate** – The percentage of connections to the same service but different destination hosts.

4. Host-Based Traffic Features (Long-Term Analysis of Connections)

These attributes monitor network behavior over a longer period to detect slow attacks:

- **dst_host_count** – The number of connections made to the same destination host.
- **dst_host_srv_count** – The number of connections made to the same service on the destination host.
- **dst_host_same_srv_rate** – The percentage of connections to the same service at the destination.
- **dst_host_diff_srv_rate** – The percentage of connections to different services on the same host.
- **dst_host_same_src_port_rate** – The percentage of connections from the same source port.
- **dst_host_srv_diff_host_rate** – The percentage of connections to different hosts but using the same service.

- **dst_host_serror_rate** – The percentage of connections to the destination host with SYN errors.
- **dst_host_srv_serror_rate** – The percentage of service connections to the host that have SYN errors.
- **dst_host_rerror_rate** – The percentage of connections to the host that result in rejection.
- **dst_host_srv_rerror_rate** – The percentage of service connections to the host that result in rejection.

5. Target Label (Attack or Normal Connection)

- **label** – The final classification of the network connection as **normal or attack**.
    - **Normal:** Indicates legitimate network activity.
    - **Attack types:** The dataset may classify them as DoS (Denial of Service), Probe, U2R (User to Root), or R2L (Remote to Local) intrusions.

Result Analysis

Fig.9.1 represents the initial interface of the Graphical User Interface (GUI) application developed for security threat detection. The GUI is designed to provide an interactive and user-friendly experience, allowing users to upload, process, and analyze network security datasets efficiently. The interface typically consists of buttons, file selection options, and visualization panels to guide users through various steps, from data uploading to model evaluation. The GUI acts as the central hub for executing the ADASYN-SPC-CNN-based detection system, making it accessible even for users with minimal coding knowledge.

Fig.9.2 demonstrates the process of uploading a dataset through the GUI. The user selects the security dataset file (e.g., in CSV format) containing labeled network traffic records with four attack categories: DoS, Probe, R2L, and U2R. The interface provides a file selection dialog, enabling users to browse and choose the dataset from their system. Once the dataset is selected, the GUI validates the file format and structure before proceeding. A status message is displayed to confirm whether the dataset has been successfully loaded or if any errors need to be addressed.

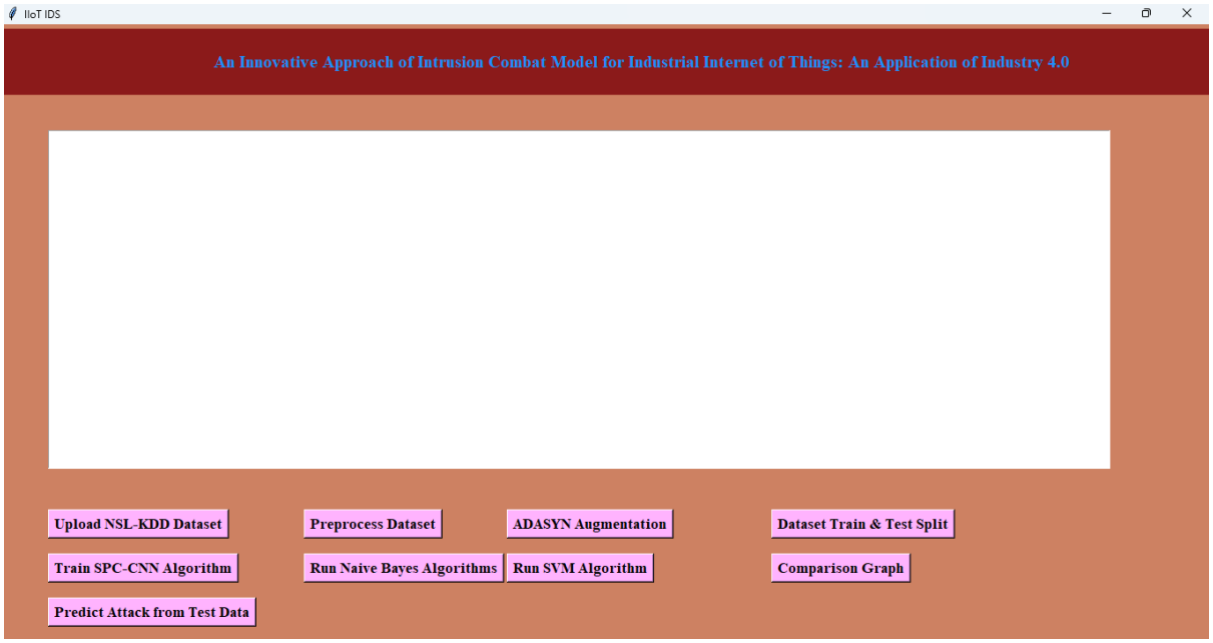


Fig. 9.1: Illustration of GUI application.

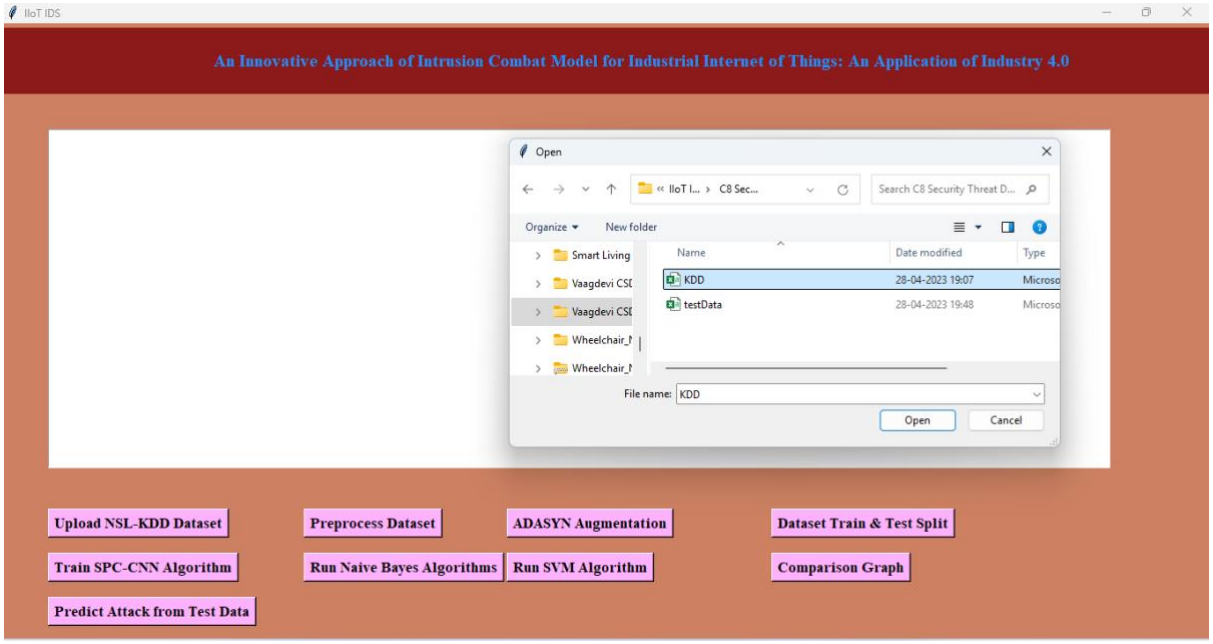


Fig. 9.2: Illustration of GUI application of uploading dataset.

In Fig.9.3, the dataset is successfully uploaded,shows the GUI displaying key details of the dataset. It presents an overview, including the number of records, features, and distribution of attack classes. The interface typically highlights the total dataset size, allowing users to verify that the data has been correctly loaded before proceeding to preprocessing and augmentation steps. At this stage, users can inspect the dataset structure to ensure its completeness and consistency before applying further operations.

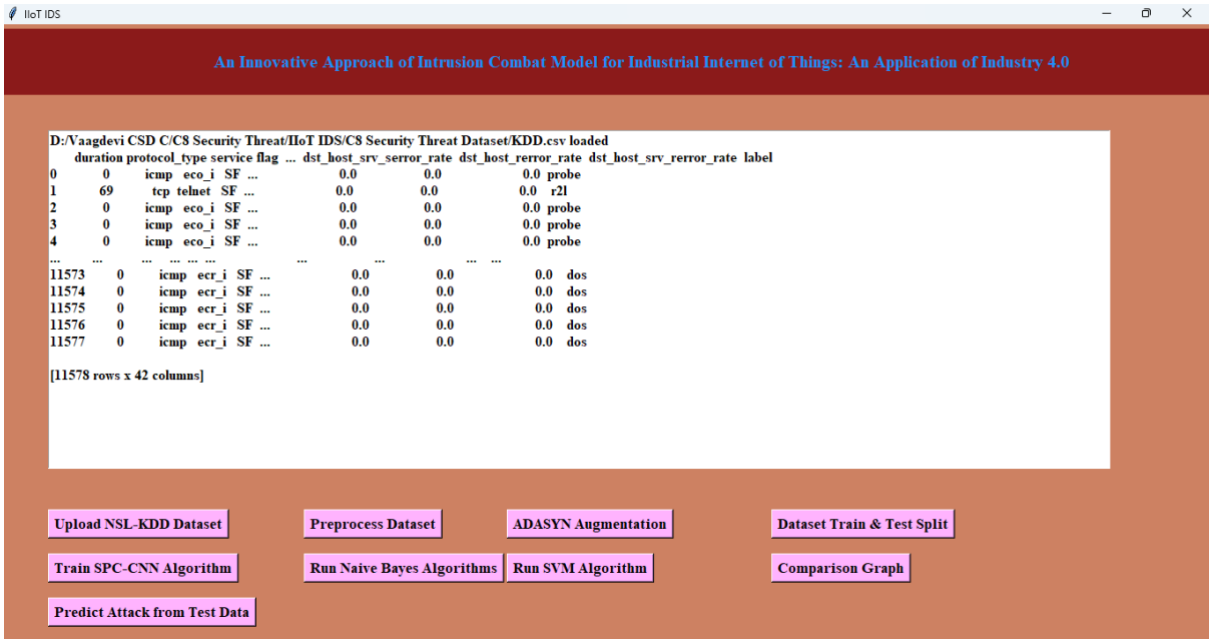


Fig. 9.3: GUI application after successful upload of the dataset.

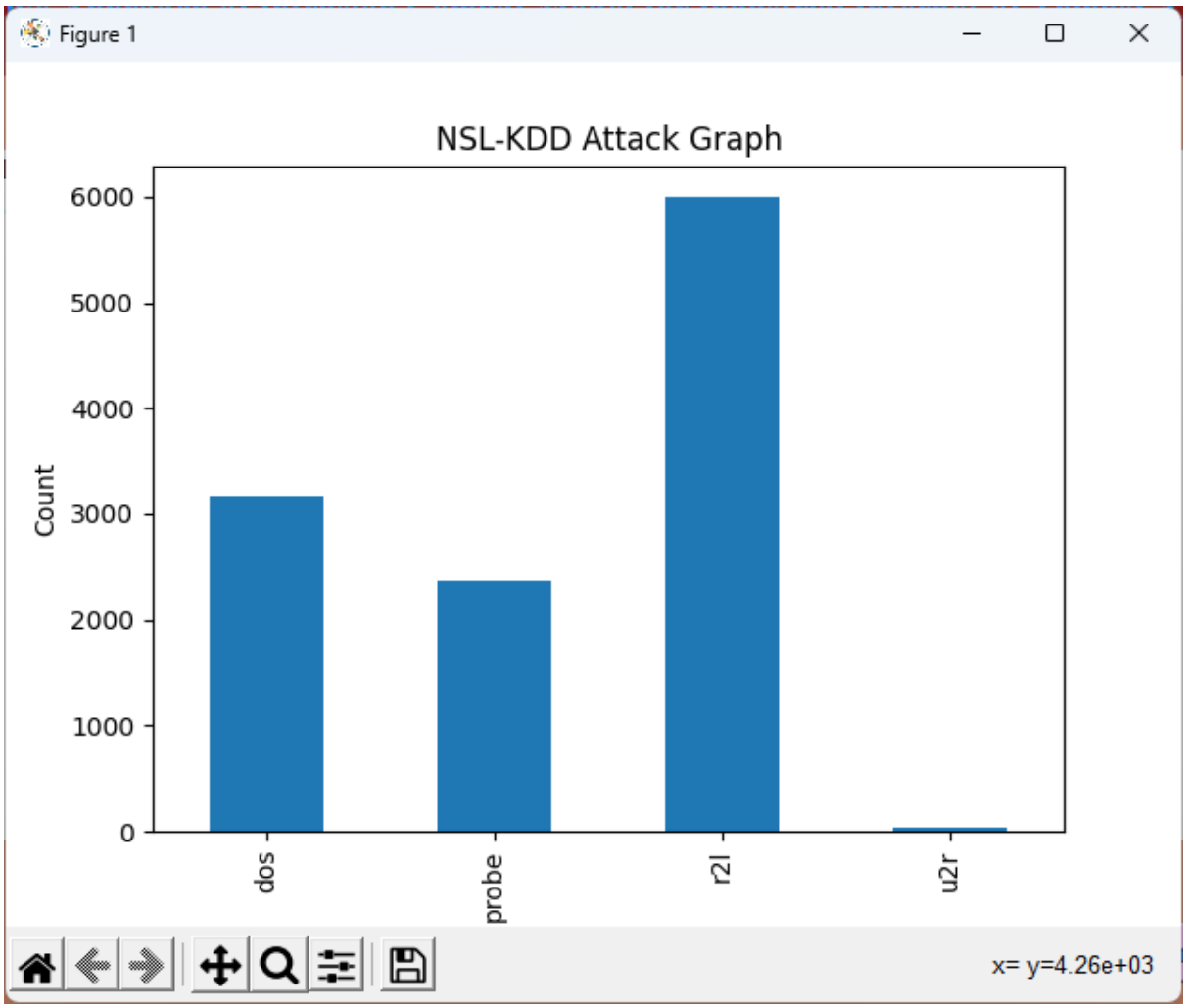


Fig. 9.4: Count plot of target variable.

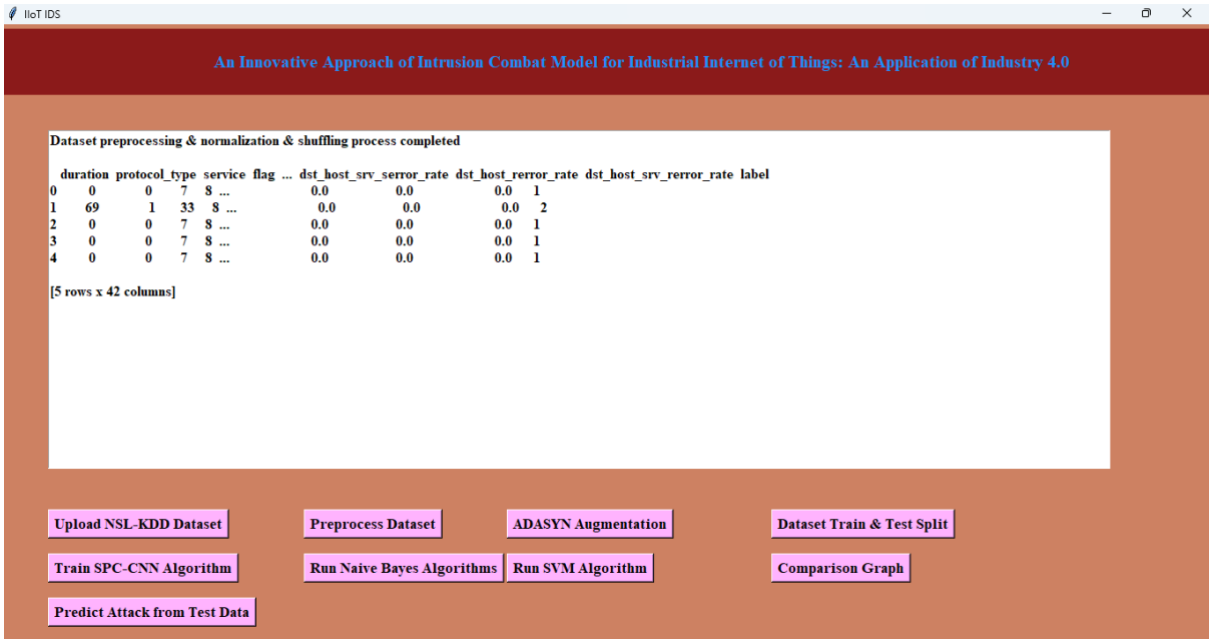


Fig. 9.5: Illustration of GUI application after dataset preprocessing.

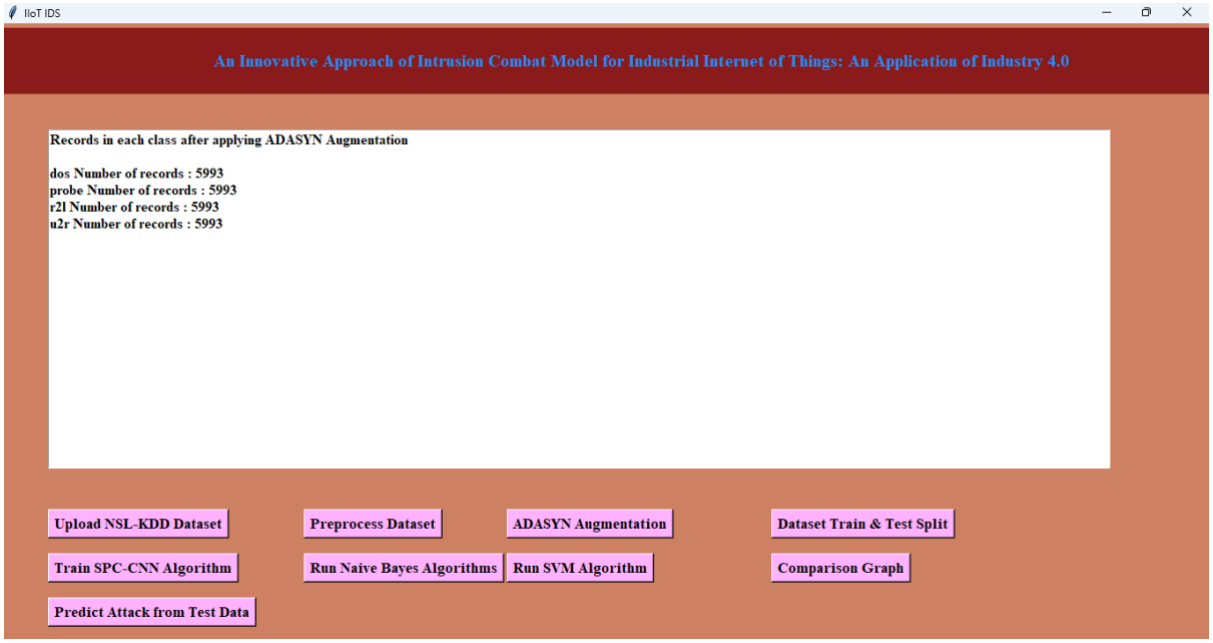


Fig. 9.6: GUI application after applying ADASYN Augmentation.

Fig.9.4 presents a visual representation of the class distribution within the dataset, specifically a count plot showing the number of records for each attack category (DoS, Probe, R2L, U2R). The count plot provides a clear understanding of data imbalance, which is crucial before applying data balancing techniques like ADASYN. In many cybersecurity datasets, attack types like DoS and Probe have significantly more records than R2L and U2R, which are rare.

This visualization helps users identify the class imbalance problem, ensuring informed decisions for data augmentation.

Fig.9.5 demonstrates the state of the GUI after data preprocessing has been applied. Preprocessing steps typically include removing missing values, encoding categorical variables, standardizing numerical features, and filtering irrelevant data. The interface displays the number of records retained after preprocessing, along with updates regarding cleaned and structured data. This step is essential to ensure that the dataset is well-prepared for model training, reducing errors and improving the accuracy of threat detection.

Fig.9.6 highlights the dataset distribution after applying the ADASYN (Adaptive Synthetic Sampling) augmentation technique.R2L and U2R attacks are typically underrepresented, ADASYN generates synthetic data points to balance the dataset, improving model performance. The GUI displays an updated count plot, showing that all attack categories now have a more even distribution of records. The balanced dataset enhances the ability of the SPC-CNN model to accurately detect both frequent and rare attack types, leading to more robust and reliable security threat classification.

Fig.9.7 represents the count plot visualization after applying ADASYN (Adaptive Synthetic Sampling) augmentation to balance the dataset. Initially, the dataset had high class imbalance, with attack types such as DoS and Probe having a significantly higher number of records than R2L and U2R. After ADASYN augmentation, the count plot now shows a more evenly distributed dataset, where all four attack classes—DoS, Probe, R2L, and U2R—have a similar number of records. This ensures that the deep learning model does not favor majority classes and instead learns to recognize patterns across all attack types, improving classification accuracy.

Fig.9.8 illustrates the GUI after splitting the dataset into training and testing sets. The train-test split process ensures that 80% of the records are used for training the model, while 20% are reserved for testing. The GUI displays the total number of records, followed by the count of records assigned to training (X_train, y_train) and testing (X_test, y_test). This step is crucial for evaluating model performance on unseen data, helping prevent overfitting and ensuring that the trained model generalizes well to real-world cybersecurity threats.

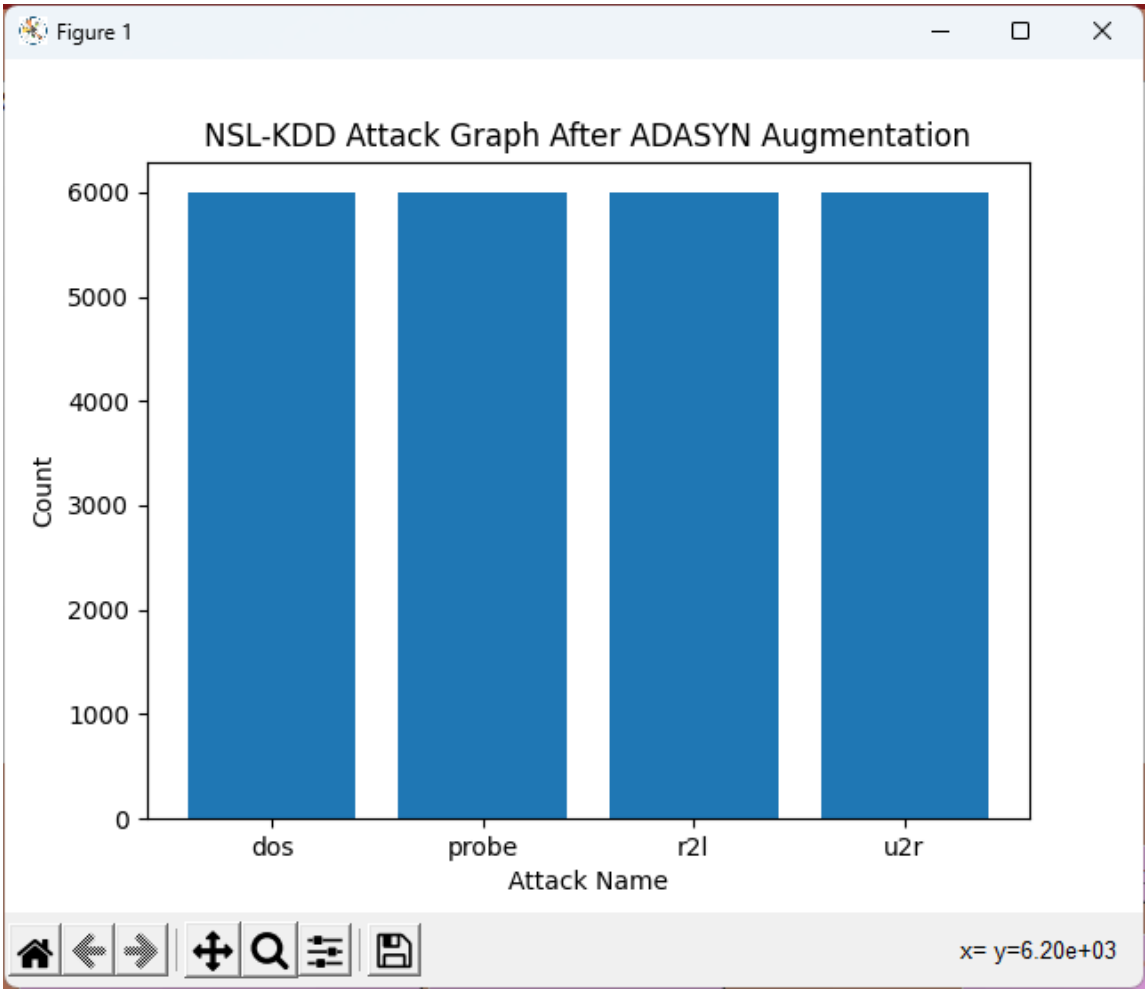


Fig. 9.7: Count plot after applying ADASYN Augmentation.

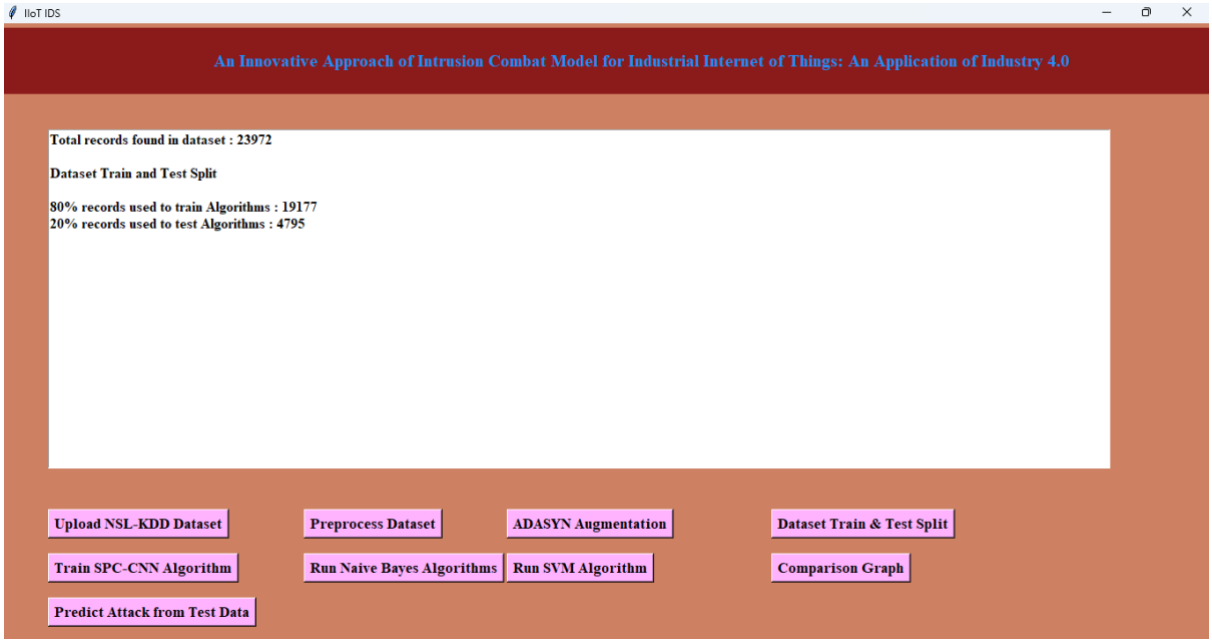


Fig. 9.8: GUI application after applying train-test splitting.

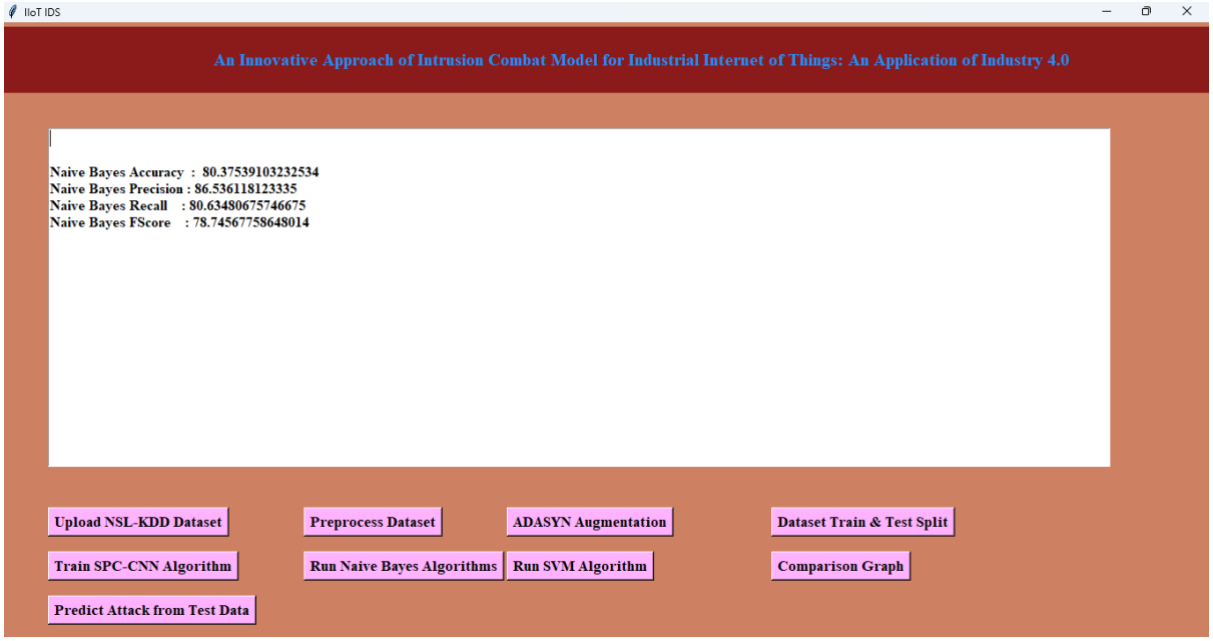


Fig. 9.9: Existing NaiveBayes performance analysis.

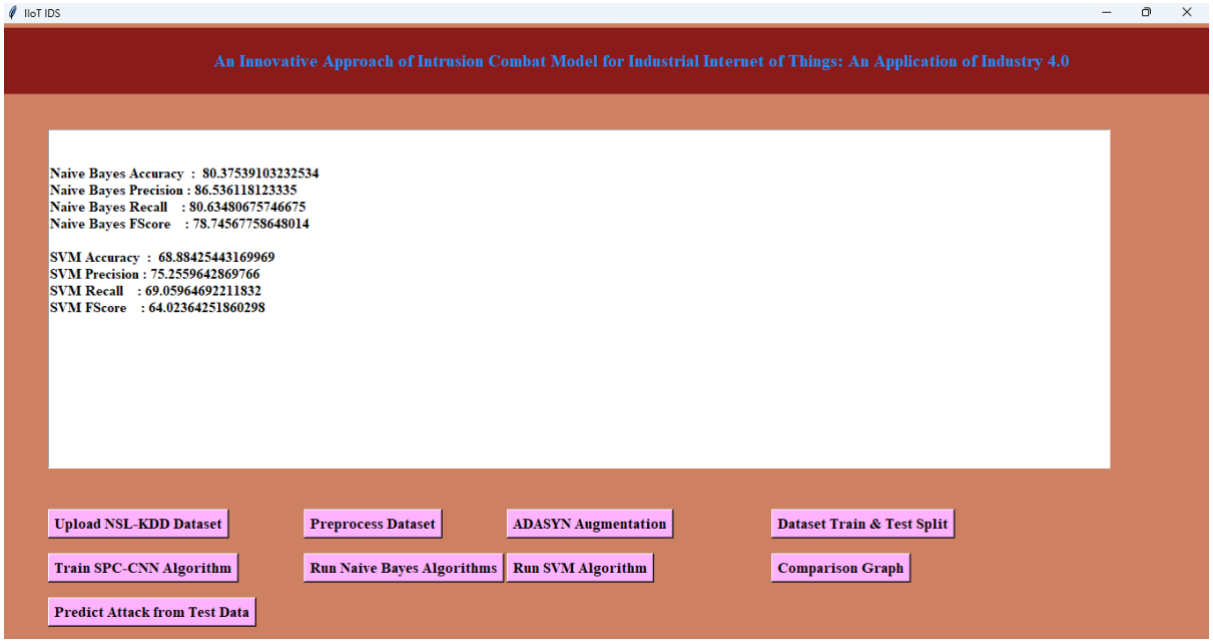


Fig. 9.10: Existing SVM performance analysis.

Fig.9.9 represents the performance evaluation of the Naïve Bayes classifier, a traditional machine learning model used for security threat detection. The model achieved:

- **Accuracy: 80.37%**, indicating that it correctly classifies threats 80.37% of the time.
- **Precision: 86.53%**, meaning that when it predicts an attack, it is correct in 86.53% of cases.

- **Recall: 80.63%**, showing that it correctly identifies 80.63% of all actual attack instances.
- **F-Score: 78.74%**, a balanced measure of precision and recall

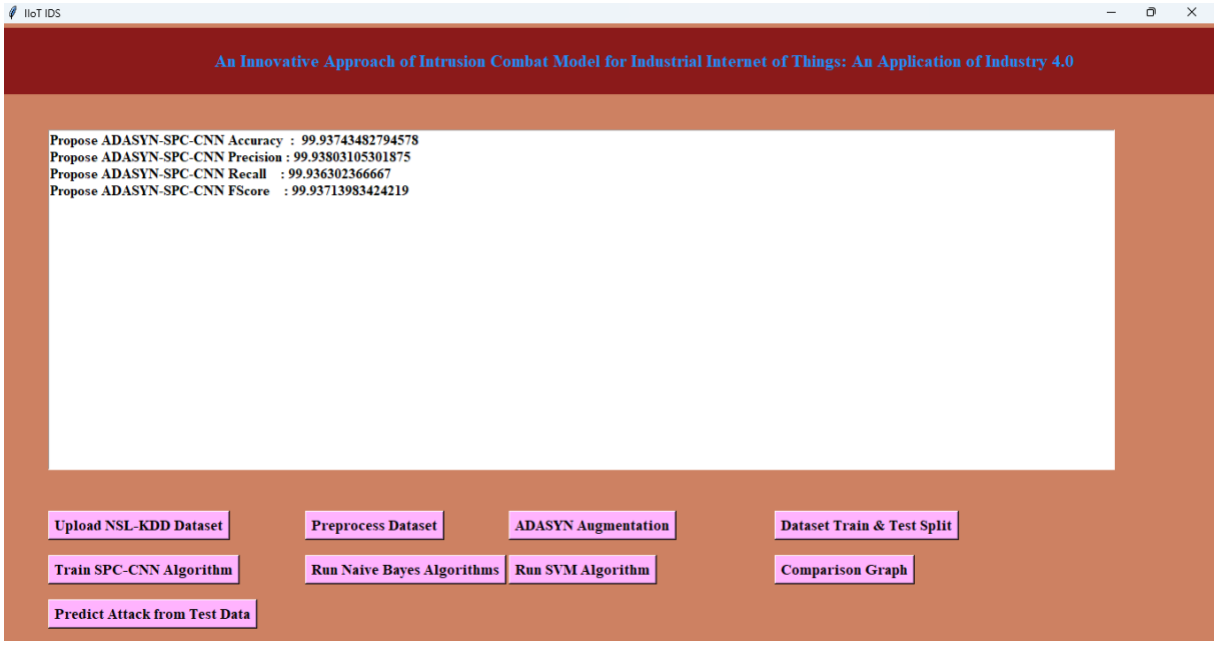


Fig.9.11: Proposed CNN model performance analysis.

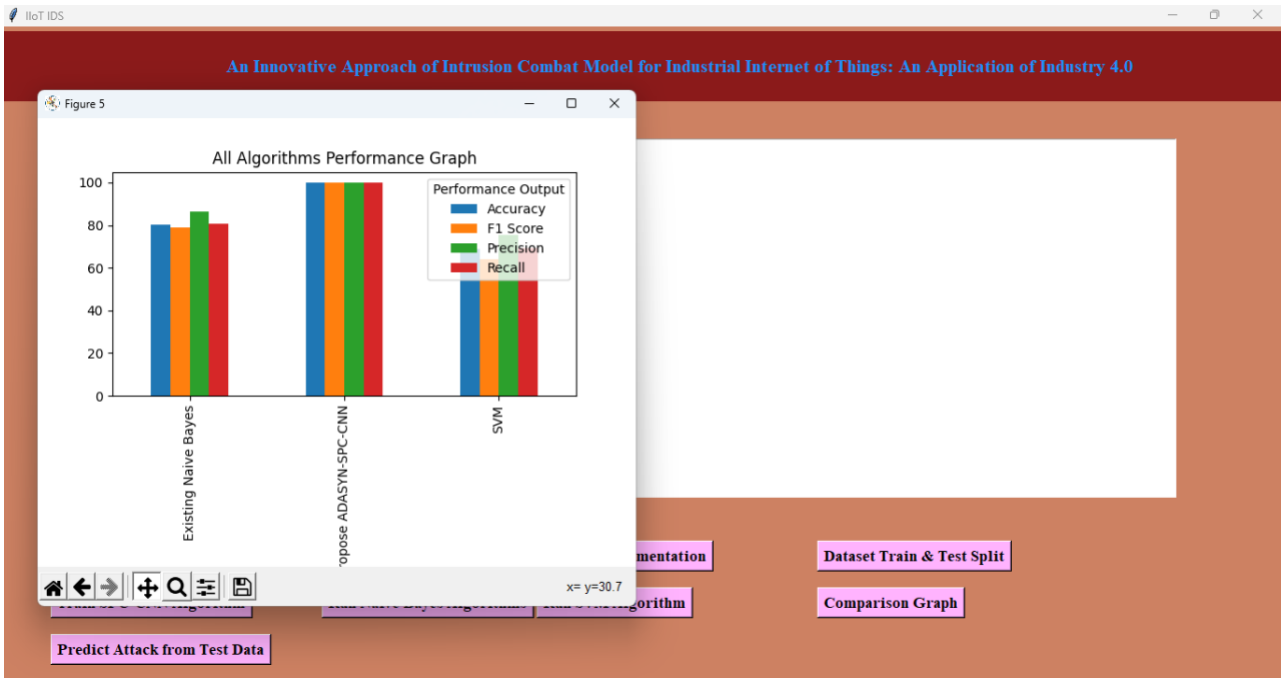


Fig. 9.12: Comparison plot of existing and proposed algorithms.

Fig.9.10 shows the Support Vector Machine (SVM) classifier's performance in detecting security threats. The model's performance metrics include:

- **Accuracy: 68.88%**, significantly lower than Naïve Bayes, indicating poor generalization.
- **Precision: 75.25%**, suggesting moderate reliability in predicting attack classes.
- **Recall: 69.05%**, indicating a lower ability to correctly identify all attack instances.
- **F-Score: 64.02%**, showing a weaker balance between precision and recall.

Fig.9.11 displays the performance of the proposed ADASYN-SPC-CNN model, which significantly outperforms existing machine learning models. The model achieves:

- **Accuracy: 99.93%**, showing near-perfect classification ability.
- **Precision: 99.93%**, meaning the model correctly predicts attack classes almost all the time.
- **Recall: 99.93%**, ensuring the model captures nearly all actual attack instances.
- **F-Score: 99.93%**, indicating excellent balance between precision and recall.

The exceptional performance of ADASYN-SPC-CNN results from its deep feature extraction capability and balanced training data, ensuring robust cybersecurity threat detection.

Fig.9.12 compares the performance of Naïve Bayes, SVM, and the proposed CNN model. The plot demonstrates that:

- Naïve Bayes performs better than SVM, but struggles with complex attack types.
- SVM has the lowest accuracy due to its computational limitations and inability to effectively classify imbalanced attack data.
- ADASYN-SPC-CNN vastly outperforms both models, achieving nearly 100% accuracy, precision, recall, and F-Score.

This comparison highlights the superiority of deep learning-based approaches in handling complex, real-world security threats.

Fig.9.13 shows the GUI interface during test data upload, where the user selects a new dataset for model evaluation. The dataset typically contains unseen attack records, which the trained ADASYN-SPC-CNN model will classify. The GUI provides a file selection prompt, allowing users to upload the test dataset, ensuring real-time security threat detection.

Fig.9.14 illustrates the final step in the security threat detection process, where the GUI displays classification results after analyzing the test dataset. The predicted attack classes (DoS, Probe, R2L, U2R) are shown alongside their respective probabilities, providing users with an interpretable and real-time threat detection report. The high accuracy of ADASYN-SPC-CNN ensures that threats are classified with confidence, making the system highly reliable for real-world cybersecurity applications.

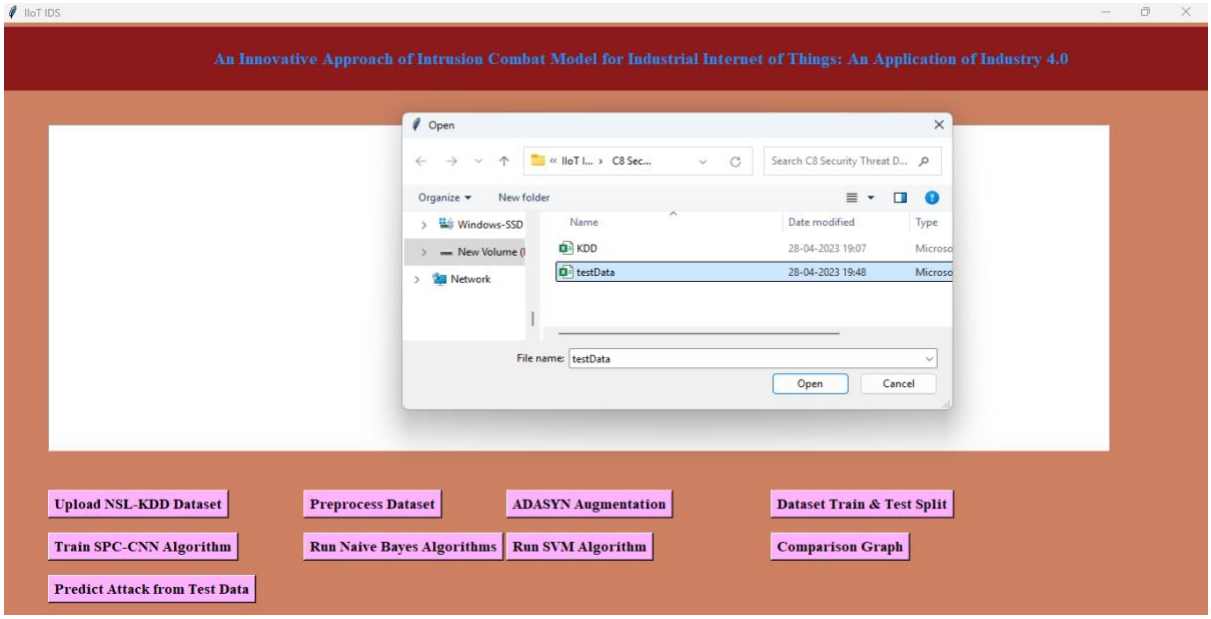


Fig. 9.13: Illustration of GUI application of uploading the test data

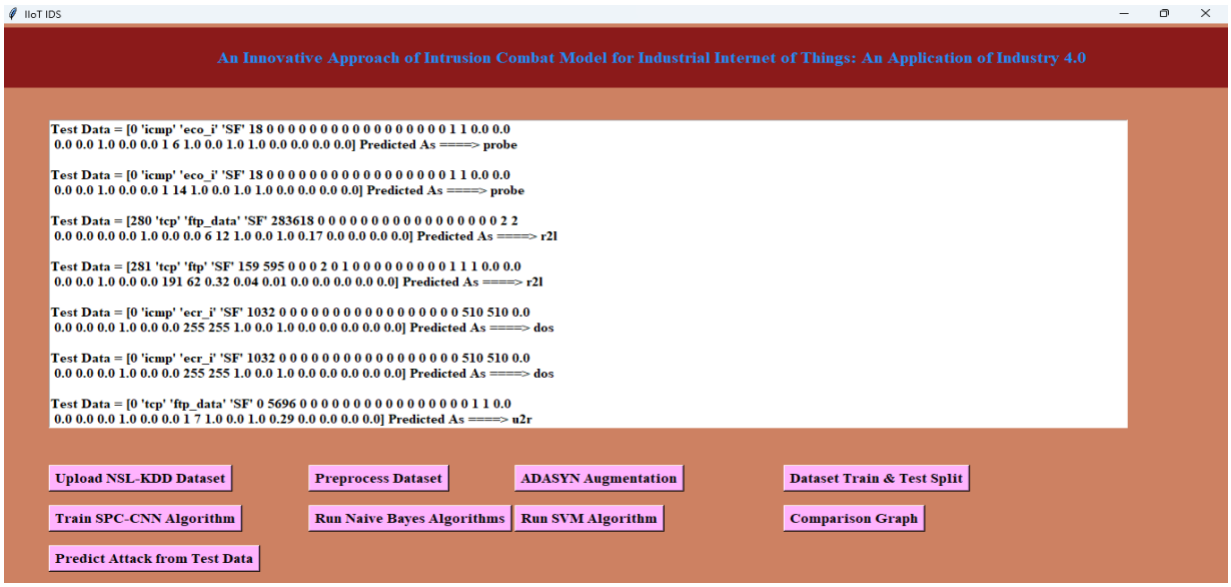


Fig. 9.14: Illustration of GUI application after successful prediction on test data.

Comparative Analysis

Table.9.1: Performance Comparison of algorithms.

Algorithm Name	Accuracy	Precision	Recall	FSCORE
Propose ADASYN-SPC-CNN	99.93743482794578	99.93803105301875	99.936302366667	99.93713983424219
Naive Bayes	80.37539103232534	86.536118123335	80.63480675746675	78.74567758648014
SVM	68.88425443169969	75.2559642869766	69.05964692211832	64.02364251860298

In Table.9.1 performance metrics across the three models show varying levels of success in their respective tasks. The Naive Bayes model achieves a balanced performance with an accuracy of 80.38%, a precision of 86.54%, and a recall of 80.63%, with an F-Score of 78.75%, indicating a good trade-off between precision and recall. The SVM model performs somewhat lower, with an accuracy of 68.88%, precision of 75.26%, recall of 69.06%, and F-Score of 64.02%, reflecting its relatively weaker performance compared to Naive Bayes. In contrast, the proposed ADASYN-SPC-CNN model significantly outperforms the others, achieving near-perfect scores across all metrics—99.94% accuracy, precision, recall, and F-Score—demonstrating its superior ability to handle the task at hand.

The confusion matrices in Fig. 9.15 compare the classification performance of three models—Support Vector Machine (SVM), Naïve Bayes, and a proposed ADASYN-SPC-CNN model—in detecting different attack categories: DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root). The SVM confusion matrix (a) shows a significant misclassification, especially in distinguishing between Probe and R2L attacks, where a notable portion of Probe instances is classified as R2L, and a similar issue is seen for other classes. The Naïve Bayes confusion matrix (b) reveals a stronger performance in classifying DoS attacks correctly but struggles considerably with Probe and R2L, showing high misclassification rates, particularly with Probe attacks being confused with R2L. Finally, the proposed ADASYN-SPC-CNN confusion matrix (c) demonstrates near-perfect classification, with minimal misclassifications across all categories. This suggests that the proposed model significantly outperforms both SVM and Naïve Bayes, achieving highly accurate attack detection with negligible errors.

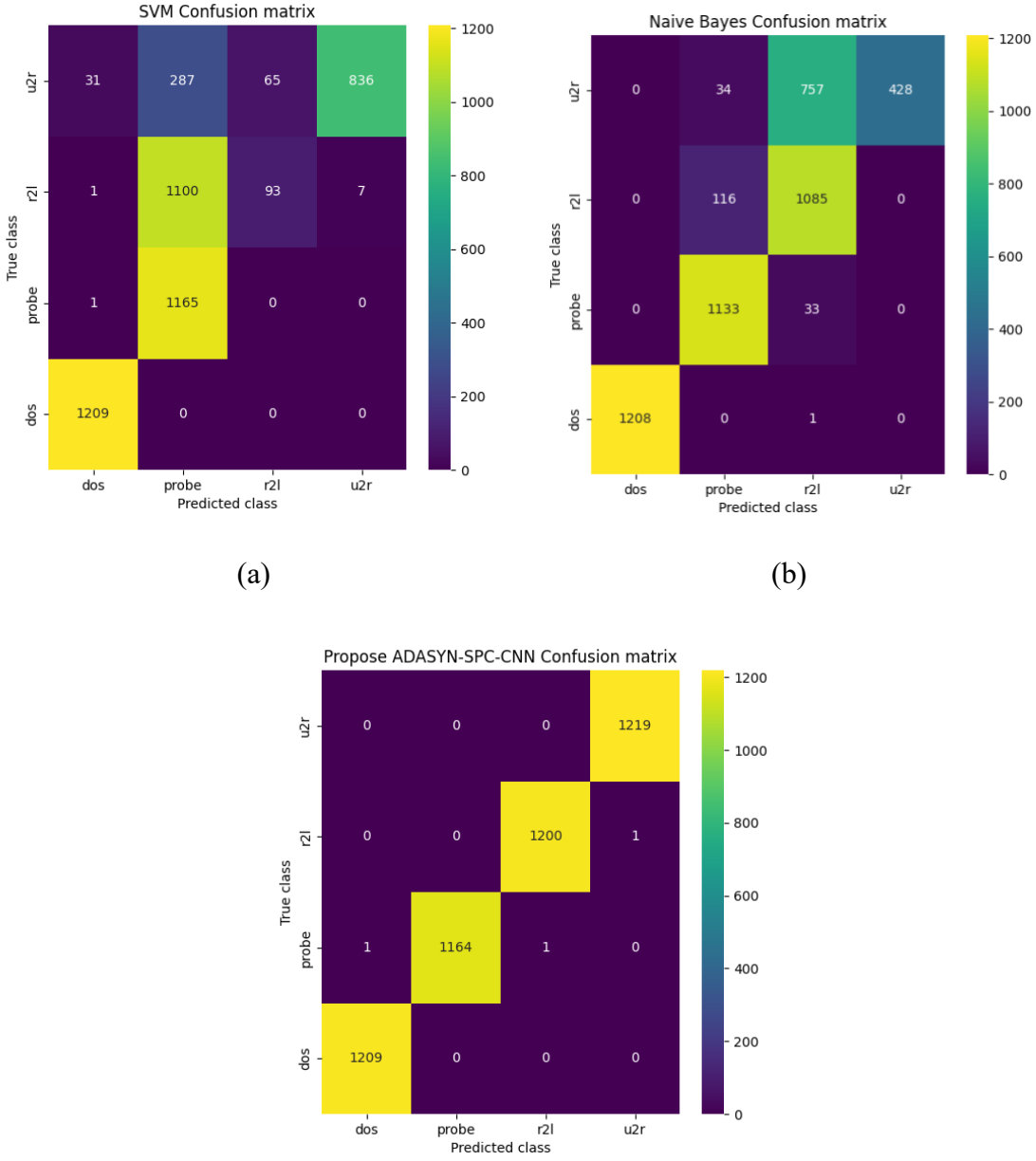


Fig. 9.15: Confusion matrices obtained using (a) SVM classifier. (b) Naïve Bayes classifier. (c) Proposed ADASYN-SPC-CNN model.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Conclusion

This research explores the application of machine learning and deep learning models—Support Vector Machine (SVM), Naïve Bayes, and a proposed ADASYN-SPC-CNN—for network intrusion detection using a dataset with multiple attack types. The dataset, consisting of various network traffic features, was analyzed using confusion matrices to evaluate classification performance. The results indicate that traditional models like SVM and Naïve Bayes struggle with imbalanced data, leading to poor detection rates for rare attack types such as R2L and U2R. In contrast, the ADASYN-SPC-CNN model significantly improves classification accuracy by addressing data imbalance through synthetic sample generation and leveraging deep learning's ability to extract complex features. The confusion matrices highlight the near-perfect detection capability of the CNN-based model, making it a robust solution for intrusion detection. This research demonstrates that combining oversampling techniques like ADASYN with deep learning can effectively mitigate class imbalance issues and enhance cybersecurity defense mechanisms.

Future Scope

The project can be extended to integrate real-time network monitoring for instant intrusion alerts and adapt to emerging threats by incorporating advanced deep learning architectures. Additionally, future work may involve multi-source data fusion and continuous learning to enhance detection accuracy and robustness in evolving IIoT environments.

BIBILOGRAPHY

[1] P.-F. Wu and H.-J. Shen, ‘‘The research and amelioration of patternmatching algorithm in intrusion detection system,’’ in Proc. IEEE 14th Int. Conf. High Perform. Comput. Commun. IEEE 9th Int. Conf. Embedded Softw. Syst., Liverpool, U.K., Jun. 2012, pp. 1712–1715, doi: 10. 1109/HPCC.2012.256.

[2] V. Dagar, V. Prakash, and T. Bhatia, ‘‘Analysis of pattern matching algorithms in network intrusion detection systems,’’ in Proc. Int. Conf. Adv. Comput., 2016, pp. 1–5.

[3] I. S. Thaseen and C. A. Kumar, ‘‘Intrusion detection model using fusion of chi-square feature selection and multi class SVM,’’ J. King Saud Univ.- Comput. Inf. Sci., vol. 29, no. 4, pp. 462–472, Oct. 2017.

[4] B. Ingre, A. Yadav, and A. K. Soni, ‘‘Decision tree based intrusion detection system for NSL-KDD dataset,’’ in Proc. Int. Conf. Inf. Commun. Technol. Intell. Syst., Ahmedabad, India, 2017, pp. 207–218, doi: 10.1007/978- 3-319-63645-0_23.

[5] P. Nancy, S. Muthurajkumar, S. Ganapathy, S. V. N. S. Kumar, M. Selvi, and K. Arputharaj, ‘‘Intrusion detection using dynamic feature selection and fuzzy temporal decision tree classification for wireless sensor networks,’’ IET Commun., vol. 14, no. 5, pp. 888–895, Mar. 2020, doi: 10. 1049/iet-com.2019.0172.

[6] M. A. Jabbar, R. Aluvalu, and S. S. Satyanarayana Reddy, ‘‘Intrusion detection system using Bayesian network and feature subset selection,’’ in Proc. IEEE Int. Conf. Comput. Intell. Comput. Res. (ICCIC), Coimbatore, India, Dec. 2017, pp. 1–5, doi: 10.1109/ICCIC.2017.8524381.

[7] T.-T.-H. Le, J. Kim, and H. Kim, ‘‘An effective intrusion detection classifier using long short-term memory with gradient descent optimization,’’ in Proc. Int. Conf. Platform Technol. Service (PlatCon), Busan, South Korea, Feb. 2017, pp. 1–6, doi: 10.1109/PlatCon.2017.7883684.

[8] T. Su, H. Sun, J. Zhu, S. Wang, and Y. Li, ‘‘BAT: Deep learning methods on network intrusion detection using NSL-KDD dataset,’’ IEEE Access, vol. 8, pp. 29575–29585, 2020, doi: 10.1109/ACCESS.2020. 2972627.

[9] P. Wei, Y. Li, Z. Zhang, T. Hu, Z. Li, and D. Liu, ‘‘An optimization method for intrusion detection classification model based on deep belief network,’’ IEEE Access, vol. 7, pp. 87593–87605, 2019, doi: 10. 1109/ACCESS.2019.2925828.

[10] M. Gao, L. Ma, H. Liu, Z. Zhang, Z. Ning, and J. Xu, ‘‘Malicious network traffic detection based on deep neural networks and association analysis,’’ Sensors, vol. 20, no. 5, p. 1452, Mar. 2020.

[11] R. Vinayakumar, K. P. Soman, and P. Poornachandran, ‘‘Applying convolutional neural network for network intrusion detection,’’ in Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI), Udupi, India, Sep. 2017, pp. 1222–1228, doi: 10.1109/ICACCI.2017.8126009.

[12] Y. Ding and Y. Zhai, ‘‘Intrusion detection system for NSL-KDD dataset using convolutional neural networks,’’ in Proc. 2nd Int. Conf. Comput. Sci. Artif. Intell. (CSAI), 2018, pp. 81–85.

[13] H. Yang and F. Wang, ‘‘Wireless network intrusion detection based on improved convolutional neural network,’’ IEEE Access, vol. 7, pp. 64366–64374, 2019, doi: 10.1109/ACCESS.2019.2917299.

[14] Q. Zhang, Z. Jiang, Q. Lu, J. Han, Z. Zeng, S.-H. Gao, and A. Men, ‘‘Split to be slim: An overlooked redundancy in vanilla convolution,’’ 2020, arXiv:2006.12085. [Online]. Available: http://arxiv.org/abs/ 2006.12085

[15] L. Dhanabal and S. P. Shantharajah, ‘‘A study on NSL-KDD dataset for intrusion detection system based on classification algorithms,’’ Int. J. Adv. Res. Comput. Commun. Eng., vol. 4, no. 6, pp. 446–452, 2015

International Journal of Communication Networks and Information Security
2025, 17(3)
E-ISSN: :2073-607X
https://https://ijcnis.org/

Research Article

Machine Learning-based Behavioural Analysis for Security Threat Detection

Sristi Lakshmi Lalitha[1*], Urukonda Shruti[2], Nagarjunapu Sai Vishwa Teja [2], Vuppula Charantej[2]

[1]Associate Professor, [2]UG Student, [1,2]Department of Computer Science and Engineering (Data Science), Vaagdevi College of Engineering, Bollikunta, Warangal, Telangana.

*Corresponding Email: lalithayagnam03@gmail.com

ARTICLE INFO

Received: 14-02-2025
Accepted: 30-03-2025

ABSTRACT

With cyber threats increasing at an alarming rate, security breaches have surged by 67% over the past five years, and cybercrime is expected to cost the world $10.5 trillion annually by 2025. Traditional manual threat detection methods rely on rule-based systems and human expertise, which are prone to errors, slow in response, and inefficient in handling large-scale data. To address these challenges, we propose a machine learning-based behavioural analysis approach for security threat detection, leveraging deep learning for improved accuracy. The proposed method begins with data preprocessing to clean and normalize the Security Threat Dataset, which contains four attack labels: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). To handle class imbalance, we employ the ADASYN (Adaptive Synthetic Sampling) technique, which generates synthetic minority class samples, ensuring a balanced dataset. The dataset is then split into training and testing sets to evaluate model performance. We compare traditional classifiers such as Naïve Bayes and Support Vector Machine (SVM) with our proposed Convolutional Neural Network (CNN)-based model, which captures complex patterns and hierarchical representations in network traffic data. Experimental results demonstrate that CNN outperforms existing methods in terms of detection accuracy, recall, and precision, making it a robust solution for real-time security threat analysis.

Keywords: Machine Learning, Behavioural Analysis, Security, Naïve Bayes, CNN, Threat Detection.

1. INTRODUCTION

The rapid development of the IIoT has brought significant growth to the global economy. However, it has also brought security risks such as leakage of core industrial data and unauthorized manipulation of interconnected terminals. Industrial firewalls are a common means of protecting the Industrial Internet of Things. However, industrial firewall technology only provides passive defense mechanisms and may not effectively prevent files and programs containing threat codes.

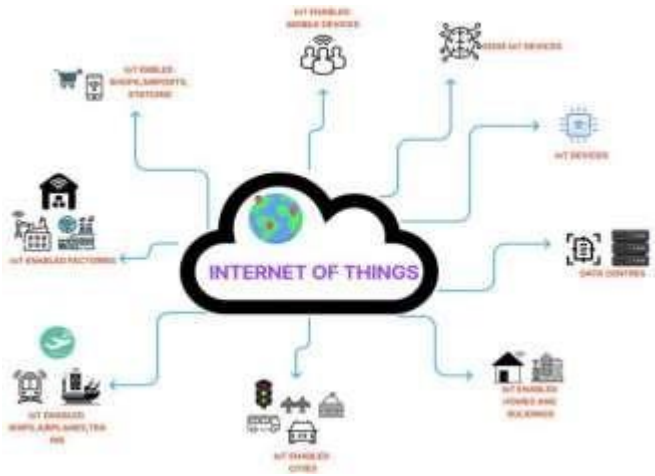


Fig. 1: Industrial IoT system.

To address these issues, intrusion detection technology has been deployed as an active network security measure. By continuously monitoring the network, it provides real-time protection against both internal and external intrusions. The characteristics of intrusion detection include proactiveness, real-timeliness, and dynamism, making it a promising research direction in the field of IIoT security. Intrusion detection technology employs classifiers to distinguish between normal and abnormal data flows, generating alerts for potential attack behaviors.

2. LITERATURE SURVEY

The development of attack recognition technology has gone through three stages: pattern matching algorithms, machine learning algorithms and deep learning algorithms. The pattern matching algorithm was first applied to intrusion detection tasks based on feature matching. In [1], Wu and Shen analyzed the classical pattern matching algorithms, BM and AC, and proposed the corresponding improved algorithms BMHS and AC-BM; experiments illustrated that the enhanced algorithms greatly optimize the timeliness of IDS. Dagar et al. [2] applied RabinKarp and Knuth-MorrisPratt pattern matching algorithms to intrusion detection tasks and compared their execution efficiency. However, these pattern matching algorithms are difficult to adapt to today's network environment due to the diversity of network attacks. An attack recognition algorithm based on ML has been successfully applied to IDS and achieved excellent performance, which gradually replaced the traditional pattern matching algorithm. SVM is a typical supervised learning model in ML.

Thaseen and Kumar [3] presented a novel attack recognition model based on chi-square feature selection and multi class support vector machine (SVM). The simulation illustrates that removing redundant features significantly improves the calculation accuracy and execution efficiency of the model. Ingre et al. [4] established a novel intrusion recognition system by combining the relevant feature screening algorithm with decision trees. Nancy et al. [5] designed a dynamic recursive feature selection algorithm. By extending the decision tree algorithm and combining it with convolutional neural networks. They proposed an intelligent fuzzy temporal decision tree algorithm. The new algorithm achieved a high detection rate of unknown attacks on the KDD cup dataset.

An improved IDS based on a Bayesian network and feature selection algorithm was proposed in [6]. Although these ML detection algorithms achieved higher recognition accuracy in intrusion detection tasks, they not only need large-scale feature engineering but the model

parameters are also difficult to adjust. However, the DL algorithm can autonomously abstract features from basic network traffic without complex feature engineering. Therefore, related research on intrusion detection is gradually focused on the DL method. An LSTM classifier with a gradient descent optimizer is used in IDS [7], which can effectively mine the association between features from the perspective of time. Su et al. [8] combined an attention mechanism and BLSTM (bidirectional long short-term memory) to propose a network anomaly detection model BAT, which extracts coarse-grained features by connecting forward LSTM and backward LSTM. The BAT model uses an attention mechanism to filter the network flow vectors generated by the BLSTM model to obtain the key characteristics of network traffic classification.

Wei et al. [9] applied particle swarm optimization (PSO) to optimize the structure of DBN, and the improved DBN achieves significant anomaly detection ability. Gao et al. [10] designed an effective attack recognition method by combining association rules and improved deep neural networks (DNN), which uses the apriori algorithm to mine the association between discrete features and labels to improve the recognition accuracy. Yin et al. [10] presented an effective attack recognition model by using feature enhancement and improved RNN; however, the feature enhancement method also increases the computational complexity of the model. CNN has been successfully applied to intrusion detection tasks because it can extract network traffic characteristics more effectively [4]. Lin et al. [5] designed a character-level CL-CNN model. The character-based encoding method makes the features more discretized, which contributes to improving the detection accuracy of IDS.

Wu et al. [11] designed a CNN model with a simple structure and proved the necessity of converting the original data into a 2D format through experiments. In addition, the combination of this simple CNN and 2D data conversion greatly improves the detection efficiency of the model compared with the RNN model in [12]. Ding and Zhai [5] proposed a convolutional neural network model (MS-CNN) based on multistage features. Multistage features are obtained by connecting the outputs of all convolutional layers to the dense layer with the softmax classifier. By adding supplementary information (such as local information and detailed information lost by higherlevel convolutional layers), the expressive ability of the model significantly improves. Yang and Wang [13] extracted diverse features through a cross-layer aggregated CNN model, which greatly improved the expression ability of the model. Although the above attack recognition algorithms using CNN improve the detection accuracy, they ignore the interchannel information redundancy in the convolution layer. However, we cannot directly discard some channel information because we are not sure which channels are redundant.

To reasonably eliminate the problem of interchannel information redundancy, Zhang et al. [14] proposed a split-based plug and play convolution (SPC) block, which divides the channel of the convolution layer into two parts, the representative part and the uncertain redundant part, after which hierarchical processing is performed. Inspired by this idea, we propose an SPC-equipped CNN (SPC-CNN) and apply it to attack recognition tasks. The simulation illustrates that the comprehensive performance of SPC-CNN has obvious advantages compared with the traditional CNN model. Before that, the ADASYN data augmentation algorithm is used to prevent the model from being sensitive to large samples and ignores small samples. Then, the AS-CNN model mixed with the ADASYN algorithm and SPC-CNN is used for intrusion detection tasks. The simulation illustrates that the comprehensive performance of the hybrid AS-CNN model is better than that of using SPC-CNN alone.

3. PROPOSED METHODOLOGY

This novel security threat detection method has not been presented in existing surveys and effectively overcomes the drawbacks of traditional approaches by integrating preprocessing, data balancing (ADASYN), train-test splitting, and deep learning (CNN). Traditional security detection methods, such as Naïve Bayes and Support Vector Machine (SVM), struggle with complex attack patterns and imbalanced datasets, leading to low detection rates for minority attacks like R2L and U2R. To overcome these issues, we propose a CNN-based approach with ADASYN for security threat detection. Our methodology first preprocesses the dataset, ensuring high-quality input data. Next, ADASYN (Adaptive Synthetic Sampling) is applied to generate synthetic minority class samples, addressing class imbalance. The dataset is then split into training and testing sets to evaluate model performance. Unlike traditional classifiers, our Convolutional Neural Network (CNN) learns deep hierarchical features from network traffic, significantly improving classification accuracy. This approach enhances real- time detection, minimizes false positives, and ensures high precision, making it a superior alternative to existing machine learning-based security systems.

Step-1: Data Preprocessing and Normalization: The first step involves cleaning and preprocessing the Security Threat Dataset, which consists of four attack labels: Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). This process includes removing duplicate, irrelevant, and inconsistent records to ensure data quality. Missing values are handled using statistical imputation techniques. Numerical features are normalized to a 0–1 scale using Min-Max scaling, which prevents bias during model training. Additionally, feature selection and dimensionality reduction may be applied to eliminate redundant features and improve overall model efficiency.

Step-2: Addressing Class Imbalance Using ADASYN: Security datasets often suffer from severe class imbalance, where minority attack types such as R2L and U2R have significantly fewer samples compared to majority classes like DoS. To address this, we use Adaptive Synthetic Sampling (ADASYN), which generates synthetic samples for minority attack classes to ensure better balance. Unlike traditional oversampling methods, ADASYN prioritizes harder-to-classify samples, thereby improving the model's learning capability. This step reduces bias toward majority classes and enhances detection performance for rare attack types.

Step-3: Train-Test Splitting for Model Evaluation: The dataset is divided into training and testing sets using an 80-20 ratio. This ensures that the model is evaluated on unseen data, which helps in assessing its real-world performance and generalization capabilities.

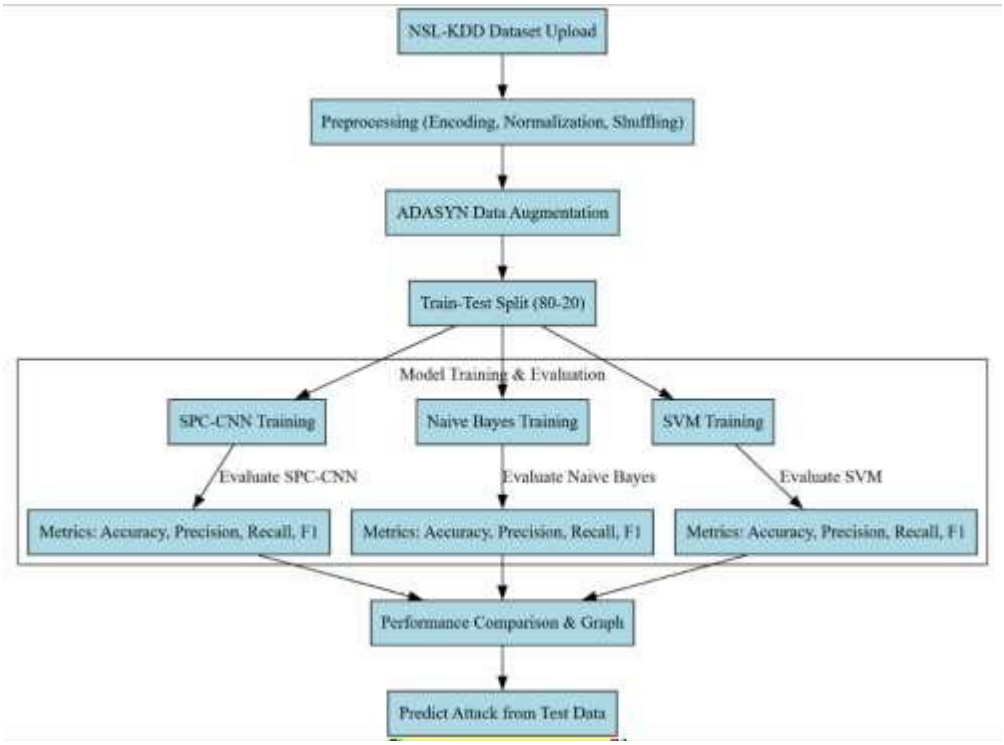


Fig. 2: Proposed block diagram of Security threat detection.

Step-4: Proposed Deep Learning Model: Convolutional Neural Network (CNN): To overcome the limitations of traditional machine learning models like Naïve Bayes and SVM, we introduce a CNN-based deep learning model for security threat detection. Unlike conventional approaches, CNNs automatically learn hierarchical features from network traffic, which significantly enhances classification performance. The model begins with an input layer that receives the preprocessed network traffic data. Convolutional layers then extract spatial and temporal features from the data, followed by pooling layers that reduce dimensionality while preserving key characteristics. Fully connected layers combine the extracted features to classify network traffic into one of the four attack types: DoS, Probe, R2L, or U2R. Finally, a Softmax activation function outputs probability scores for each class, enabling accurate classification.

Step-5: Performance Evaluation: The model is evaluated using key classification metrics including accuracy, precision, recall, and F1-score. Accuracy measures the overall correctness of the model. Precision and recall assess the model's ability to correctly detect attacks while minimizing false positives. The F1-score provides a balance between precision and recall, ensuring robust classification across all attack categories.

3.2 Data Balancing with ADASYN (Adaptive Synthesis)

In machine learning, particularly in cybersecurity threat detection, datasets are often highly imbalanced. This means that some attack classes (e.g., DoS) may have thousands of records, while rarer attacks like User-to-Root (U2R) or Remote-to-Local (R2L) may have only a few instances. Class imbalance poses a major challenge because machine learning models tend to be biased toward majority classes, leading to poor detection rates for minority attack types. Standard classification algorithms may ignore rare attacks since they contribute less to the training error, resulting in an increased rate of false negatives. To address this, oversampling techniques are used to artificially increase the number of samples in the minority classes. The given code applies ADASYN (Adaptive Synthetic Sampling) to balance the dataset by

generating synthetic data points for underrepresented attack classes, making the dataset more uniform and improving model performance.

The ADASYN object is defined by creating a sampling model. Though often SMOTE is used (sm = SMOTE (random_state=2)), this should be replaced with ADASYN if adaptive sampling is the goal. The fit_sample(X, Y) function is then applied to synthesize new samples for the underrepresented classes in the dataset.ADASYN computes the density distribution of the minority classes and generates synthetic samples primarily for those that are hardest to classify. This makes it more targeted than traditional oversampling, as it doesn't treat all minority class samples equally. By focusing on challenging cases, ADASYN enhances the learning process and overall model robustness. Once applied, the dataset now reflects a more balanced distribution of attack classes. This distribution can be confirmed by counting the records in each class using np.unique(Y, return_counts=True), showing the effect of data augmentation.

Visualization of the new class balance is essential. A bar chart is created with attack types on the x-axis and sample counts on the y-axis, illustrating how the synthetic samples have leveled the distribution. The chart title is updated to reflect post-augmentation data and is displayed with plt.show() for easy inspection.

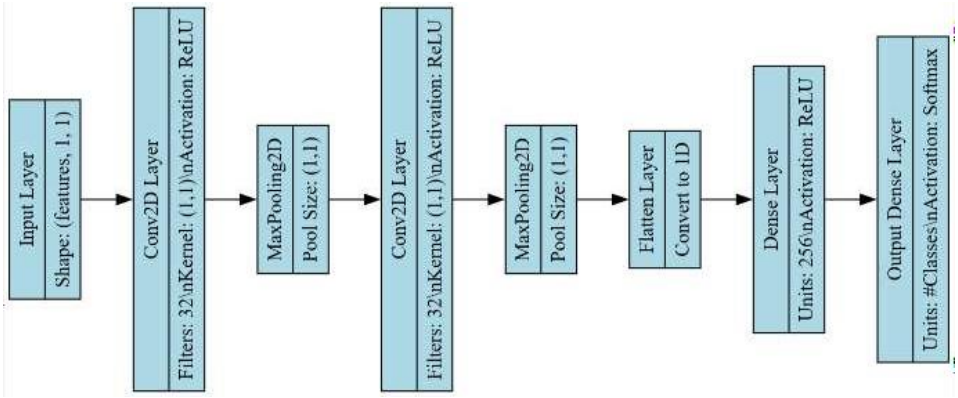


Fig. 4: Layered architecture of proposed deep learning model.

3.3 Proposed CNN model

Convolutional Neural Networks (CNNs) are a deep learning architecture capable of recognizing complex patterns, making them particularly suitable for cybersecurity threat detection. CNNs automatically extract relevant features from raw data through a series of convolutional and pooling operations, reducing reliance on manual feature engineering. CNN layers apply filters to input data to detect meaningful spatial and temporal patterns, especially in structured formats like network logs. These features are then refined through pooling layers, which discard noise and redundancy. Finally, the output is passed to fully connected layers that perform classification using a softmax activation function. In threat detection, CNNs can learn intricate attack signatures, classifying traffic into DoS, Probe, R2L, and U2R with high accuracy. Their ability to generalize and adapt makes them superior to traditional machine learning models.SPC-CNN (Specialized CNN) is trained with X_train and y_train and later evaluated using X_test and y_test. Input data is reshaped into a format compatible with CNNs. Labels are converted to categorical format using one-hot encoding. The convolutional layers extract features using multiple filters, followed by max-pooling to down sample and retain key information. The process is repeated for deeper feature learning. After flattening the feature maps, the model passes them through dense layers. The final output layer, using softmax,

classifies the data into one of the four attack types. The model is compiled with the Adam optimizer and trained using categorical cross-entropy loss. If no pre-trained weights are available, the model is trained for 20 epochs, with the best model saved. After training, predictions are made on the test set and evaluated using accuracy, precision, recall, and F1-score.

4. RESULTS AND DISCUSSION

4.1 Dataset description

The dataset used in this research is the NSL-KDD dataset, which is widely utilized for security threat detection. Below is a detailed explanation of the columns included in the dataset.

Basic Features (Network Traffic Attributes): These attributes describe fundamental characteristics of network connections. "Duration" refers to the length of the connection in seconds. "Protocol_type" denotes the protocol used for the connection, such as TCP, UDP, or ICMP. "Service" indicates the network service on the destination, for example, HTTP, FTP, or SSH. "Flag" is the status flag of the connection, which shows whether the attempt was successful or failed. "Src_bytes" is the number of data bytes sent from the source to the destination, while "dst_bytes" represents the number of data bytes sent from the destination back to the source.

Content Features (Intrusion-Specific Indicators): These attributes analyze the payload of the packets to detect suspicious activity. "Land" is a binary value (0 or 1) that indicates whether the source and destination IPs and ports are the same, potentially signaling a loop attack. "Wrong_fragment" is the number of wrong or fragmented packets in the connection. "Urgent" shows the number of urgent packets sent, which are commonly used in DoS attacks. "Hot" counts indicators such as login failures or execution of system commands. "Num_failed_logins" represents the number of failed login attempts and is useful for detecting brute-force attacks. "Logged_in" is a binary value indicating whether the login attempt was successful (1) or failed (0). "Num_compromised" counts how many compromised conditions were detected in the connection. "Root_shell" is a binary indicator of whether root- level access was obtained. "Su_attempted" refers to the number of times a 'su' command (used for privilege escalation) was attempted. "Num_root" records the number of root-level commands executed. "Num_file_creations" counts the number of file creation operations during the connection. "Num_shells" indicates the number of shell sessions that were opened. "Num_access_files" captures the number of file access operations. "Num_outbound_cmds" shows the number of outbound commands in an FTP session. "Is_host_login" is a binary indicator that specifies whether the login attempt was made on the host machine. "Is_guest_login" indicates if the login was performed using a guest account.

Time-Based Traffic Features (Connection Behavior Over Time): These features capture connection activity within a specific time window. "Count" is the number of connections from the same source IP during the timeframe. "Srv_count" indicates the number of connections to the same service within the same period. "Serror_rate" and "srv_serror_rate" represent the percentage of connections and service connections, respectively, that encountered SYN errors, which could indicate a possible DoS attack. "Rerror_rate" and "srv_rerror_rate" are the percentage of connections with REJ (reject) errors, both overall and per service. "Same_srv_rate" shows the percentage of connections to the same service, whereas "diff_srv_rate" measures the percentage to different services. "Srv_diff_host_rate" reflects the percentage of connections to the same service but different destination hosts.

Host-Based Traffic Features (Long-Term Analysis of Connections): These attributes help detect long-term or slow attacks. "Dst_host_count" is the number of connections made to the same destination host. "Dst_host_srv_count" represents the number of connections to the same service on the destination host. "Dst_host_same_srv_rate" is the percentage of connections to the same service at that host, and "dst_host_diff_srv_rate" shows the percentage to different services on the same host. "Dst_host_same_src_port_rate" measures the percentage of connections from the same source port. "Dst_host_srv_diff_host_rate" records the percentage of connections to different hosts using the same service. "Dst_host_serror_rate" and "dst_host_srv_serror_rate" represent the percentage of connections and service connections to the host that result in SYN errors. "Dst_host_rerror_rate" and "dst_host_srv_rerror_rate" indicate the percentage of connections and service connections to the host that result in rejection.

Target Label (Attack or Normal Connection): The "Label" column provides the final classification of each network connection as either normal or an attack. A normal label indicates legitimate network activity. Attack types are categorized into four main groups: DoS (Denial of Service), Probe, U2R (User to Root), and R2L (Remote to Local) intrusions.

4.2 Results description

Fig. 5 represents a visual representation of the class distribution within the dataset, specifically a count plot showing the number of records for each attack category (DoS, Probe, R2L, U2R). The count plot provides a clear understanding of data imbalance, which is crucial before applying data balancing techniques like ADASYN. In many cybersecurity datasets, attack types like DoS and Probe have significantly more records than R2L and U2R, which are rare.

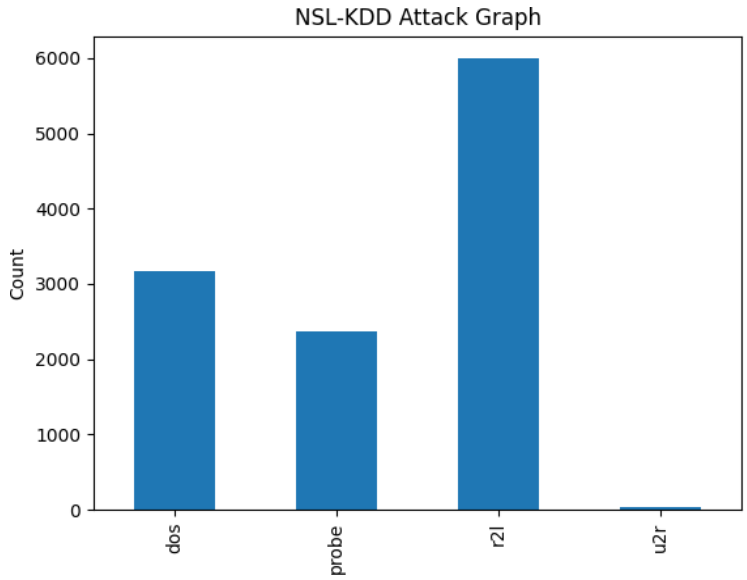


Fig. 5: Count plot of target variable before applying ADASYN Augmentation.

Fig. 6 represents the count plot visualization after applying ADASYN (Adaptive Synthetic Sampling) augmentation to balance the dataset. Initially, the dataset had high class imbalance, with attack types such as DoS and Probe having a significantly higher number of records than R2L and U2R. After ADASYN augmentation, the count plot now shows a more evenly distributed dataset, where all four attack classes—DoS, Probe, R2L, and U2R—have a similar number of records. This ensures that the deep learning model does not favor majority classes

and instead learns to recognize patterns across all attack types, improving classification accuracy.

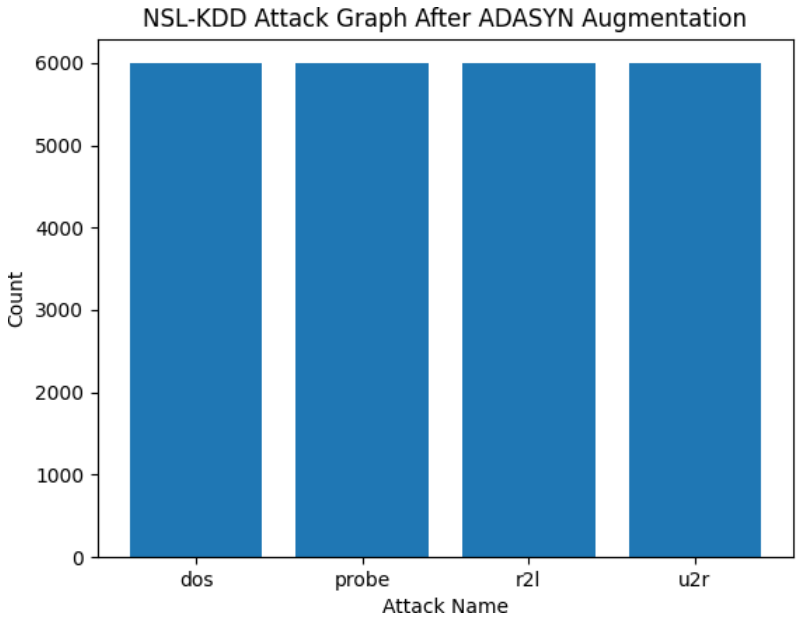


Fig. 6: Count plot after applying ADASYN Augmentation.

Fig. 7 illustrates the final step in the security threat detection process, where the GUI displays classification results after analyzing the test dataset. The predicted attack classes (DoS, Probe, R2L, U2R) are shown alongside their respective probabilities, providing users with an interpretable and real-time threat detection report. The high accuracy of ADASYN-SPC-CNN ensures that threats are classified with confidence, making the system highly reliable for real-world cybersecurity applications.

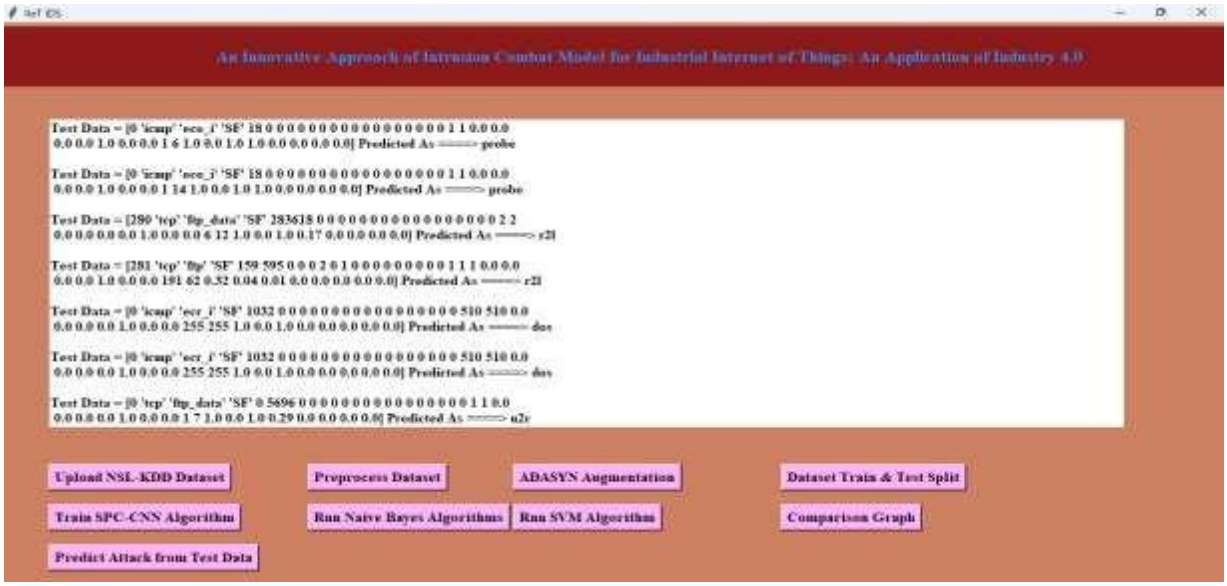
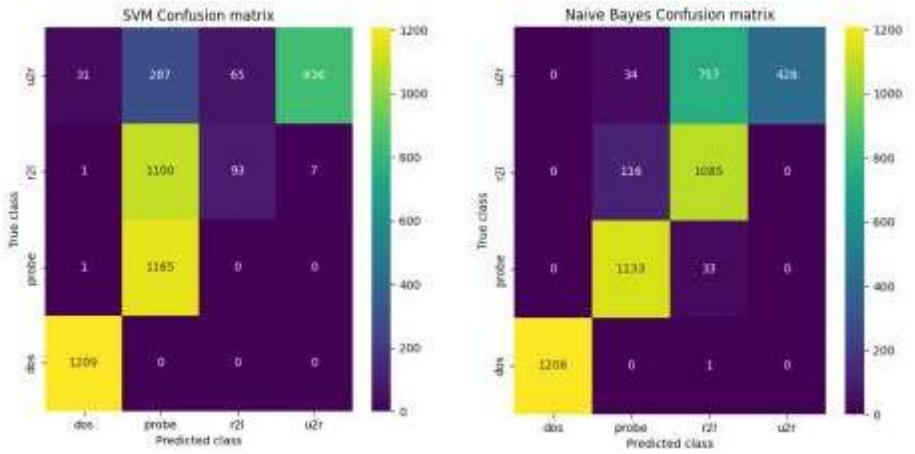


Fig. 7: Illustration of GUI application after successful prediction on test data.

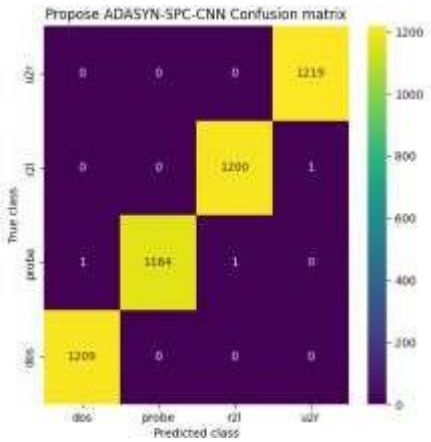
Table.1: Performance Comparison of algorithms.

Algorithm Name	Accuracy	Precision	Recall	FSCORE
Propose ADASYN-SPC-CNN	99.93743482794578	99.93803105301875	99.936302366667	99.93713983424219
Naive Bayes	80.37539103232534	86.536118123335	80.63480675746675	78.74567758648014
SVM	68.88425443169969	75.2559642869766	69.05964692211832	64.02364251860298

In Table.9.1 performance metrics across the three models show varying levels of success in their respective tasks. The Naive Bayes model achieves a balanced performance with an accuracy of 80.38%, a precision of 86.54%, and a recall of 80.63%, with an F-Score of 78.75%, indicating a good trade-off between precision and recall. The SVM model performs somewhat lower, with an accuracy of 68.88%, precision of 75.26%, recall of 69.06%, and F-Score of 64.02%, reflecting its relatively weaker performance compared to Naive Bayes. In contrast, the proposed ADASYN-SPC-CNN model significantly outperforms the others, achieving near-perfect scores across all metrics—99.94% accuracy, precision, recall, and F-Score—demonstrating its superior ability to handle the task at hand.



(a) (b)



(c)

Fig. 8: Confusion matrices obtained using (a) SVM classifier. (b) Naïve Bayes classifier. (c) Proposed ADASYN-SPC-CNN model.

The confusion matrices in Fig. 8 compare the classification performance of three models—Support Vector Machine (SVM), Naïve Bayes, and a proposed ADASYN-SPC-CNN model—in detecting different attack categories: DoS (Denial of Service), Probe, R2L (Remote to Local), and U2R (User to Root). The SVM confusion matrix (a) shows a significant misclassification,

especially in distinguishing between Probe and R2L attacks, where a notable portion of Probe instances is classified as R2L, and a similar issue is seen for other classes. The Naïve Bayes confusion matrix (b) reveals a stronger performance in classifying DoS attacks correctly but struggles considerably with Probe and R2L, showing high misclassification rates, particularly with Probe attacks being confused with R2L. Finally, the proposed ADASYN-SPC-CNN confusion matrix (c) demonstrates near-perfect classification, with minimal misclassifications across all categories. This suggests that the proposed model significantly outperforms both SVM and Naïve Bayes, achieving highly accurate attack detection with negligible errors.

5. CONCLUSION

This research explores the application of machine learning and deep learning models—Support Vector Machine (SVM), Naïve Bayes, and a proposed ADASYN-SPC-CNN—for network intrusion detection using a dataset with multiple attack types. The dataset, consisting of various network traffic features, was analyzed using confusion matrices to evaluate classification performance. The results indicate that traditional models like SVM and Naïve Bayes struggle with imbalanced data, leading to poor detection rates for rare attack types such as R2L and U2R. In contrast, the ADASYN-SPC-CNN model significantly improves classification accuracy by addressing data imbalance through synthetic sample generation and leveraging deep learning's ability to extract complex features. The confusion matrices highlight the near-perfect detection capability of the CNN-based model, making it a robust solution for intrusion detection. This research demonstrates that combining oversampling techniques like ADASYN with deep learning can effectively mitigate class imbalance issues and enhance cybersecurity defense mechanisms.

REFERENCES

[1] P.-F. Wu and H.-J. Shen, "The research and amelioration of patternmatching algorithm in intrusion detection system," in Proc. IEEE 14th Int. Conf. High Perform. Comput. Commun. IEEE 9th Int. Conf. Embedded Softw. Syst., Liverpool, U.K., Jun. 2012, pp. 1712–1715, doi: 10. 1109/HPCC.2012.256.

[2] V. Dagar, V. Prakash, and T. Bhatia, "Analysis of pattern matching algorithms in network intrusion detection systems," in Proc. Int. Conf. Adv. Comput., 2016, pp. 1–5.

[3] I. S. Thaseen and C. A. Kumar, "Intrusion detection model using fusion of chi-square feature selection and multi class SVM," J. King Saud Univ.- Comput. Inf. Sci., vol. 29, no. 4, pp. 462–472, Oct. 2017.

[4] B. Ingre, A. Yadav, and A. K. Soni, "Decision tree based intrusion detection system for NSL-KDD dataset," in Proc. Int. Conf. Inf. Commun. Technol. Intell. Syst., Ahmedabad, India, 2017, pp. 207–218, doi: 10.1007/978- 3-319-63645-0_23.

[5] P. Nancy, S. Muthurajkumar, S. Ganapathy, S. V. N. S. Kumar, M. Selvi, and K. Arputharaj, "Intrusion detection using dynamic feature selection and fuzzy temporal decision tree classification for wireless sensor networks," IET Commun., vol. 14, no. 5, pp. 888–895, Mar. 2020, doi: 10. 1049/iet-com.2019.0172.

[6] M. A. Jabbar, R. Aluvalu, and S. S. Satyanarayana Reddy, "Intrusion detection system using Bayesian network and feature subset selection," in Proc. IEEE Int. Conf. Comput. Intell. Comput. Res. (ICCIC), Coimbatore, India, Dec. 2017, pp. 1–5, doi: 10.1109/ICCIC.2017.8524381.

[7] T.-T.-H. Le, J. Kim, and H. Kim, "An effective intrusion detection classifier using long short-term memory with gradient descent optimization," in Proc. Int. Conf. Platform Technol. Service (PlatCon), Busan, South Korea, Feb. 2017, pp. 1–6, doi: 10.1109/PlatCon.2017.7883684.

[8] T. Su, H. Sun, J. Zhu, S. Wang, and Y. Li, "BAT: Deep learning methods on network intrusion detection using NSL-KDD dataset," IEEE Access, vol. 8, pp. 29575–29585, 2020, doi: 10.1109/ACCESS.2020. 2972627.

[9] P. Wei, Y. Li, Z. Zhang, T. Hu, Z. Li, and D. Liu, "An optimization method for intrusion detection classification model based on deep belief network," IEEE Access, vol. 7, pp. 87593–87605, 2019, doi: 10. 1109/ACCESS.2019.2925828.

[10] M. Gao, L. Ma, H. Liu, Z. Zhang, Z. Ning, and J. Xu, "Malicious network traffic detection based on deep neural networks and association analysis," Sensors, vol. 20, no. 5, p. 1452, Mar. 2020.

[11] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Applying convolutional neural network for network intrusion detection," in Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI), Udupi, India, Sep. 2017, pp. 1222–1228, doi: 10.1109/ICACCI.2017.8126009.

[12] Y. Ding and Y. Zhai, "Intrusion detection system for NSL-KDD dataset using convolutional neural networks," in Proc. 2nd Int. Conf. Comput. Sci. Artif. Intell. (CSAI), 2018, pp. 81–85.

[13] H. Yang and F. Wang, "Wireless network intrusion detection based on improved convolutional neural network," IEEE Access, vol. 7, pp. 64366–64374, 2019, doi: 10.1109/ACCESS.2019.2917299.

[14] Q. Zhang, Z. Jiang, Q. Lu, J. Han, Z. Zeng, S.-H. Gao, and A. Men, "Split to be slim: An overlooked redundancy in vanilla convolution," 2020, arXiv:2006.12085. [Online]. Available: http://arxiv.org/abs/ 2006.12085

[15] L. Dhanabal and S. P. Shantharajah, "A study on NSL-KDD dataset for intrusion detection system based on classification algorithms," Int. J. Adv. Res. Comput. Commun. Eng., vol. 4, no. 6, pp. 446–452, 2015.

Project Completion Certificate

This is to certify that

NAGARJUNAPU SAI VISHWA TEJA

a stufient of B. Tech CSE (Data Science) from VAAGDEVI COLLEGE OF ENGINEERING (UGC -Autonomous) locatefi at Boolikunta, Warangal has successfully completefi Major Project Training Program on

MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION

We at SAK INFORMATICS wish him/her the best anfi success in his/her life anfi career.

Student ID: 21641A67D4

Mahesh Pala
Founder & CEO
SAK Informatics

SAK INFORMATICS
Plot 544, Flat 102, Shiv Sai Murali Ramaneeyam, Peacock Circle, Pragathi Nagar, Hyderabad-90
www.sakinformatics.com ceo@sakinformatics.com (0)-9000188676, (0)-9603999243

Project Completion Certificate

This is to certify that

URUKONDA SHRUTI

a stufient of B. Tech CSE (Data Science) from VAAGDEVI COLLEGE OF ENGINEERING (UGC -Autonomous) locatefi at Boolikunta, Warangal has successfully completefi Major Project Training Program on

MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION

We at SAK INFORMATICS wish him/her the best anfi success in his/her life anfi career.

Student ID: 21641A67G6

Mahesh Pala
Founder & CEO
SAK Informatics

SAK INFORMATICS
Plot 544, Flat 102, Shiv Sai Murali Ramaneeyam, Peacock Circle, Pragathi Nagar, Hyderabad-90
www.sakinformatics.com ceo@sakinformatics.com (0)-9000188676, (0)-9603999243

Project Completion Certificate

This is to certify that

VUPPULA CHARANTEJ

a stufient of B. Tech CSE (Data Science) from VAAGDEVI COLLEGE OF ENGINEERING (UGC -Autonomous) locatefi at Boolikunta, Warangal has successfully completefi Major Project Training Program on

MACHINE LEARNING-BASED BEHAVIOURAL ANALYSIS FOR SECURITY THREAT DETECTION

We at SAK INFORMATICS wish him/her the best anfi success in his/her life anfi career.

Student ID: 21641A67J5

Mahesh Pala
Founder & CEO
SAK Informatics

SAK INFORMATICS
Plot 544, Flat 102, Shiv Sai Murali Ramaneeyam, Peacock Circle, Pragathi Nagar, Hyderabad-90
www.sakinformatics.com ceo@sakinformatics.com (0)-9000188676, (0)-9603999243